

Exhaustive Search

Chapter 4

Outline

- Restriction Enzymes
- Gel Electrophoresis
- Partial Digest Problem
- Brute Force Algorithm for Partial Digest Problem
- Branch and Bound Algorithm for Partial Digest Problem

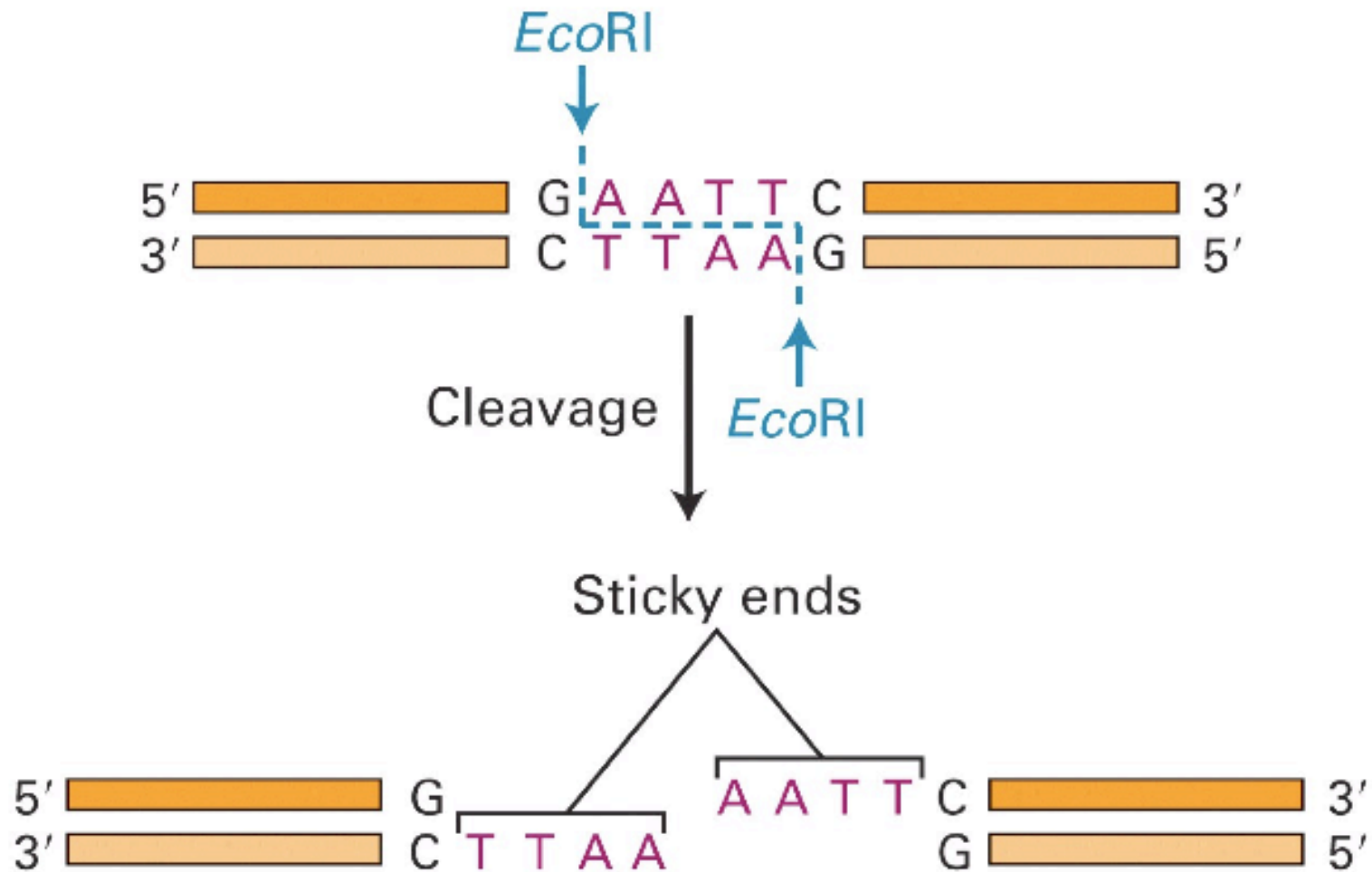
Discovering Restriction Enzymes

- *Hind II*: first restriction enzyme discovered by Hamilton Smith in 1970
 - From *Haemophilus influenzae* (bacterium sometimes associated with influenza virus infections, causes pneumonia and meningitis)
 - Discovered accidentally while studying how the bacterium *Haemophilus influenzae* takes up DNA from the phage virus P22
 - Recognizes and cuts DNA at sequences:
 - GTGCAC
 - GTTAAC
 - Breaking a long molecule into a set of *restriction fragments*

Restriction maps

- Powerful research tools in molecular biology by helping to narrow the location of certain genetic markers.

Molecular Scissors



Recognition Sites of Restriction Enzymes

Enzyme	Source Microorganism	Recognition Site *	Ends Produced
BamHI	<i>Bacillus amyloliquefaciens</i>	↓ -G-G-A-T-C-C- -C-C-T-A-G-G- ↑	Sticky
EcoRI	<i>Escherichia coli</i>	↓ -G-A-A-T-T-C- -C-T-T-A-A-G- ↑	Sticky
HindIII	<i>Haemophilus influenzae</i>	↓ -A-A-G-C-T-T- -T-T-C-G-A-A- ↑	Sticky
KpnI	<i>Klebsiella pneumonia</i>	↓ -G-G-T-A-C-C- -C-C-A-T-G-G- ↑	Sticky

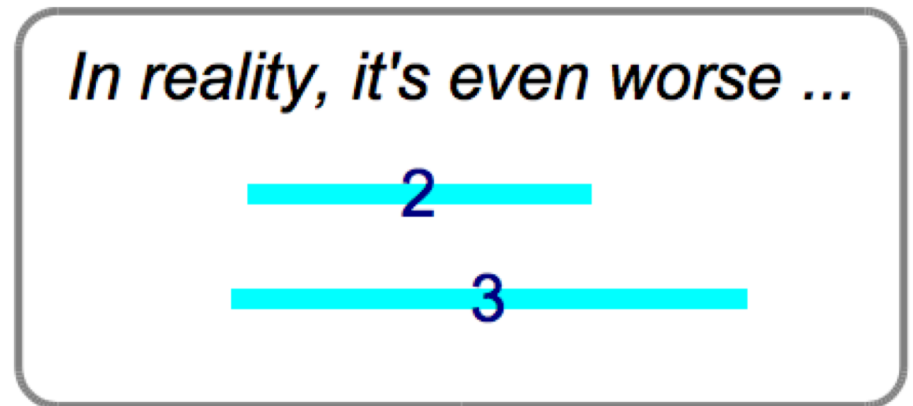
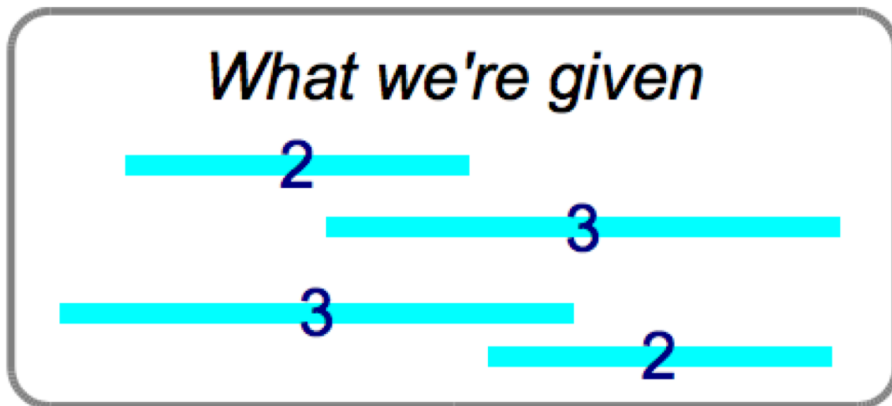
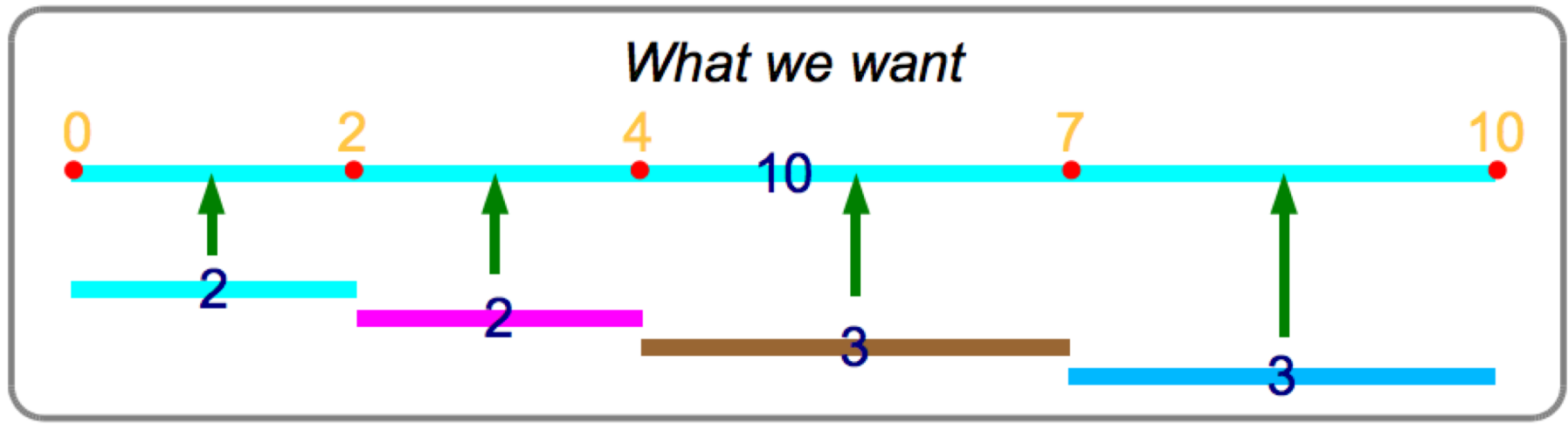
Full Restriction Digest

- Cutting DNA at each restriction site creates multiple restriction fragments:

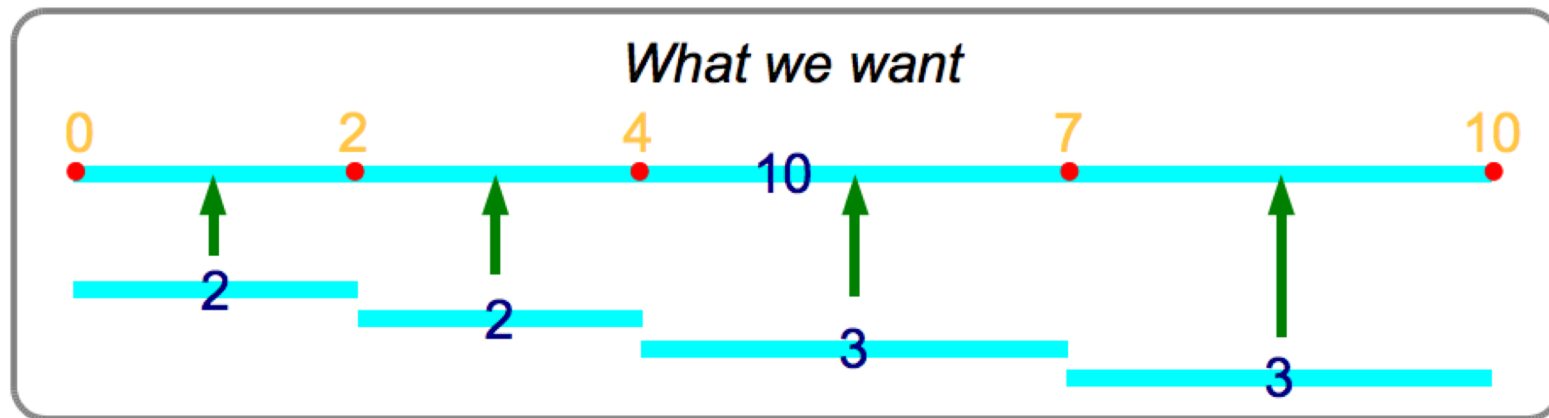


- Is it possible to reconstruct the order of the fragments and the positions of the cuts?

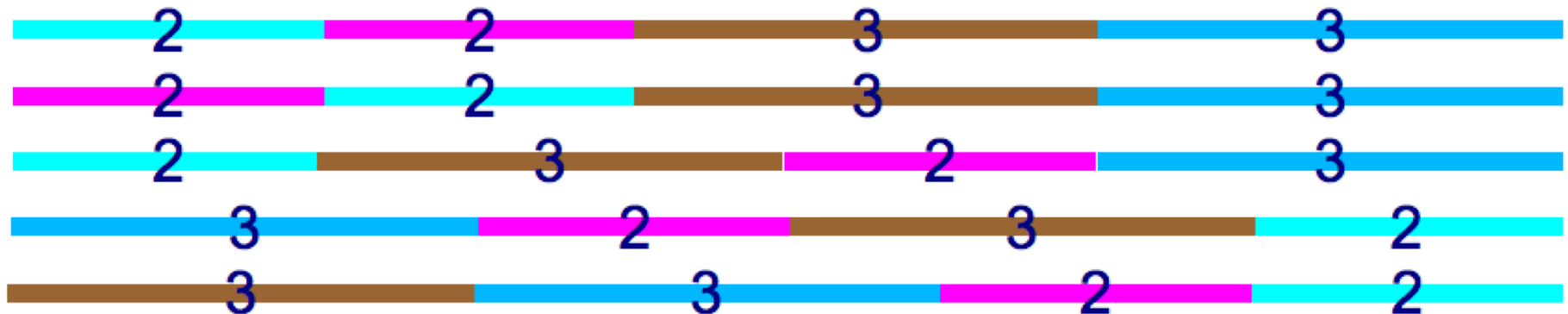
Full Restriction Digest: Multiple Solutions



Full Restriction Digest: Multiple Solutions



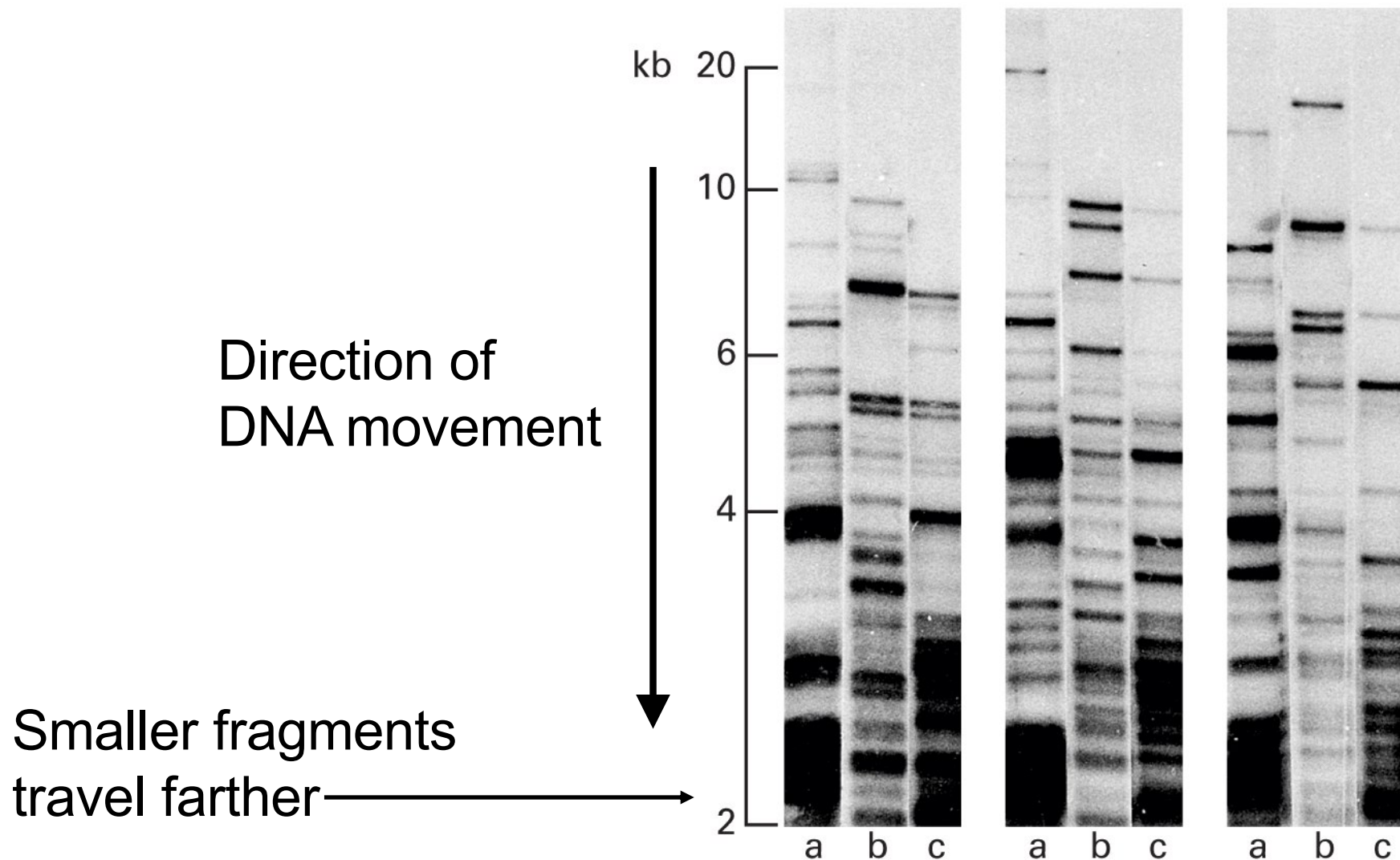
- Even if we somehow knew there were two different fragments of length 2 and 3, there are still a large number of solutions:



Separating DNA by Size

- **Gel electrophoresis** is a process for separating DNA by size
- Can separate DNA fragments that differ in length in only 1 nucleotide for fragments up to 500 nucleotides long

Gel Electrophoresis: Example



Double Digest Mapping

- Double Digest is yet another experimentally method to construct restriction maps
 - Use two restriction enzymes; three **full** digests:
 - One with only first enzyme
 - One with only second enzyme
 - One with both enzymes
 - Computationally, Double Digest problem is more complex than Partial Digest problem
-

Double Digest Problem

Input: dA – fragment lengths from the digest with enzyme A .

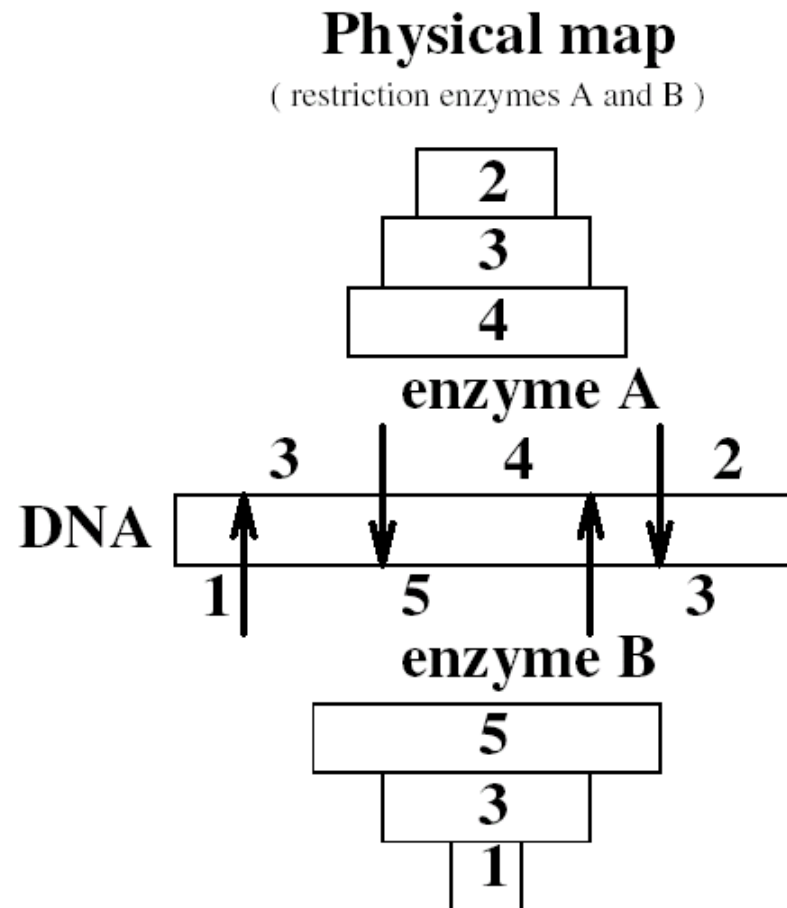
dB – fragment lengths from the digest with enzyme B .

dX – fragment lengths from the digest with *both* A and B .

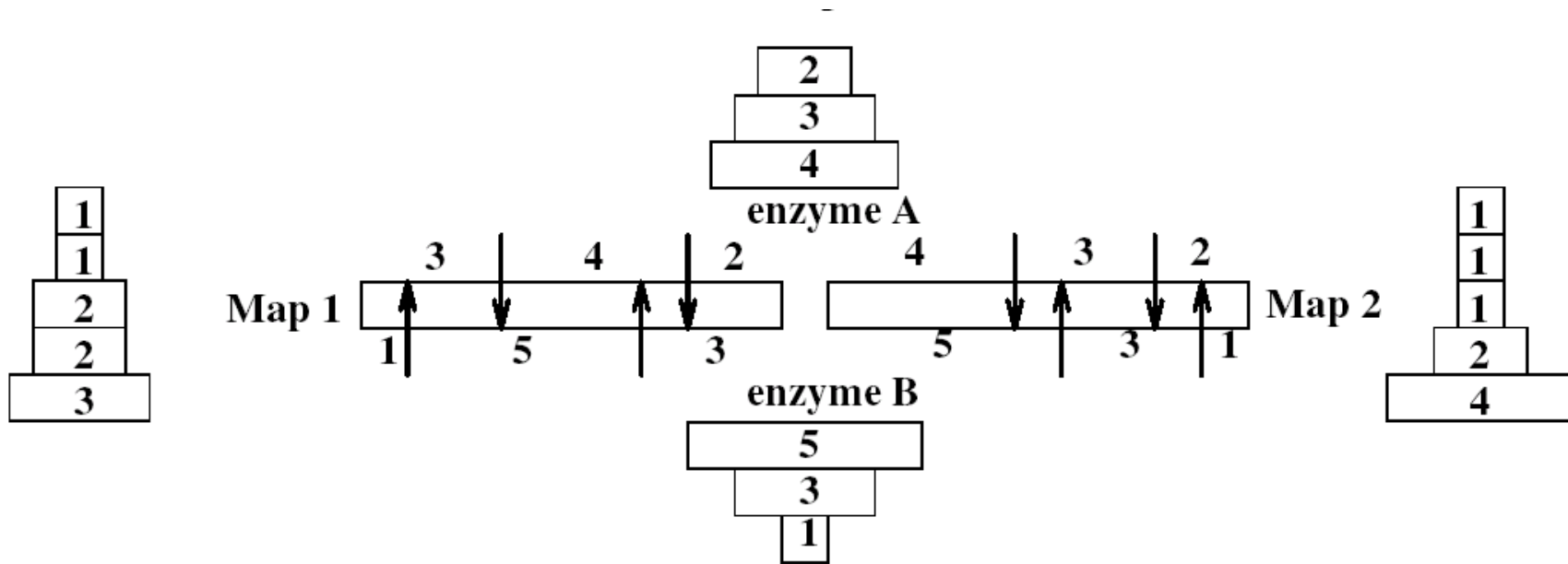
Output: A – location of the cuts in the restriction map for the enzyme A .

B – location of the cuts in the restriction map for the enzyme B .

Double Digest: Example

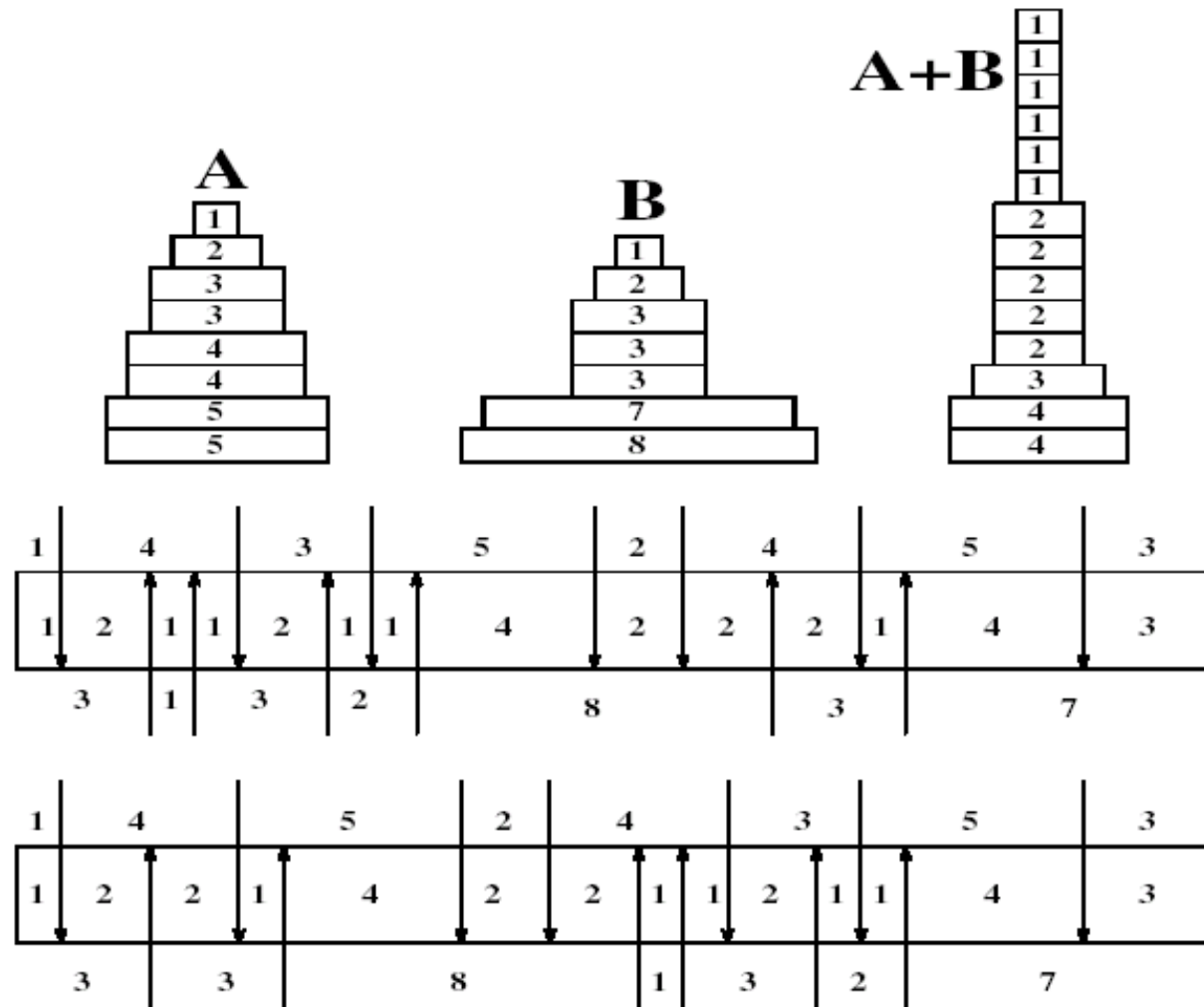


Double Digest: Example



Without the information about X (i.e. $A+B$), it is impossible to solve the double digest problem as this diagram illustrates

Double Digest: Multiple Solutions



DDP is NP-Complete

- The Set Partitioning Problem (SPP)
 - Given a (multi-)set of integers $X = \{x_1, \dots, x_n\}$. The SSP is to determine whether we can partition X into two subsets X_1 and X_2 such that

$$\sum_{x \in X_1} x = \sum_{x \in X_2} x$$

DDP is NP-Complete

- Let X be the input of the SPP, assuming that the sum of all elements of X is even. Then set
 - $\Delta A = X$,
 - $\Delta B = \left\{ \frac{K}{2}, \frac{K}{2} \right\}$. with $K = \sum_{i=1}^n x_i$, and
 - $\Delta AB = \Delta A$.
- then there exists an integer n_0 and indices $\{j_1, j_2, \dots, j_n\}$ with

$$\sum_{i=1}^{n_0} x_{j_i} = \sum_{i=n_0+1}^n x_{j_i}$$

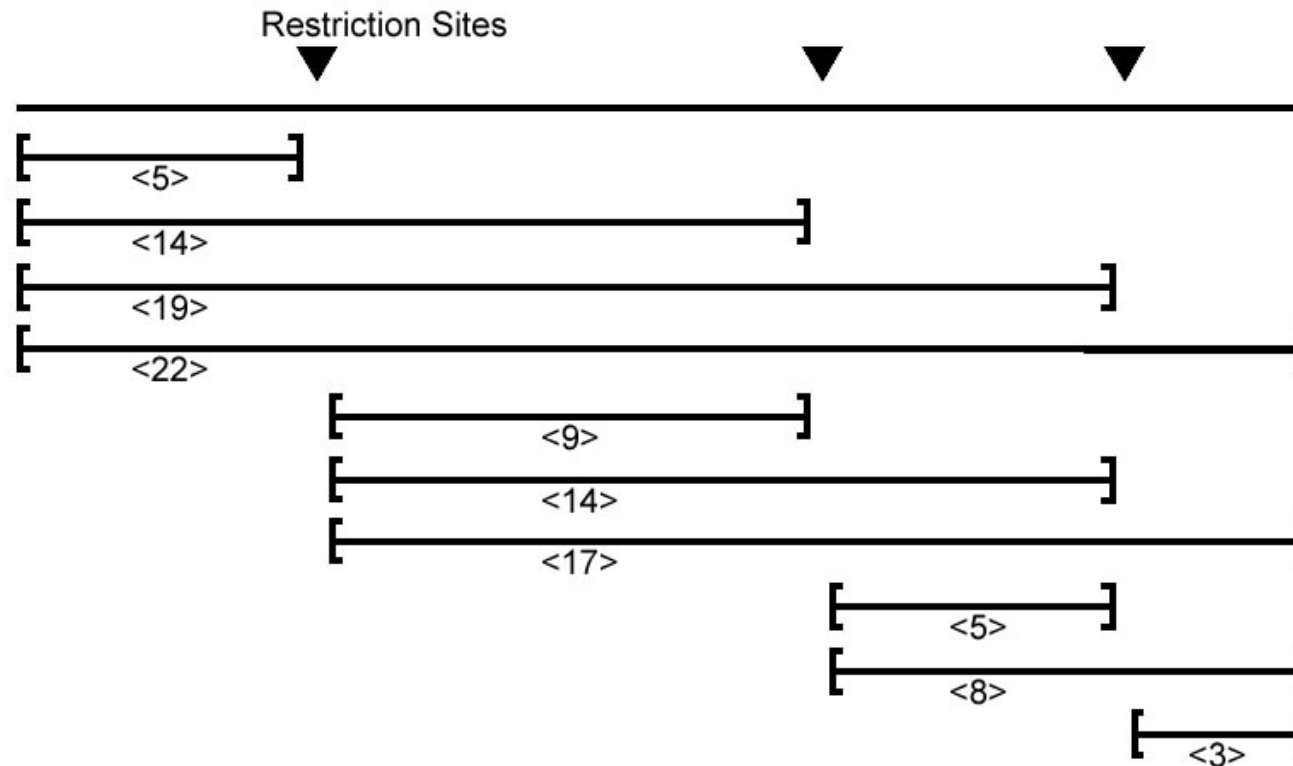
because of the choice of ΔB and ΔAB . Thus a solution for the SPP exists. Thus SPP is a DDP in which one of the two enzymes produced only two fragments of equal length.

Partial Restriction Digest

- The sample of DNA is exposed to the restriction enzyme for only a **limited amount** of time to prevent it from being cut at all restriction sites
- We assume that with this method biologists can generate the set of all possible restriction fragments between every two cuts
- We assume that multiplicity of a fragment can be detected, i.e., the number of restriction fragments of the same length can be determined (e.g., by observing twice as much fluorescence intensity for a double fragment than for a single fragment)
- This set of fragment sizes is used to determine the positions of the restriction sites in the DNA sequence

Partial Digest Example

- For the same DNA sequence as before, we would now get the following restriction fragments:



Partial Digest Fundamentals

Defining some of the terms used in the Partial Digest Problem:

X : the set of **n** integers representing the location of all cuts in the restriction map, including the start and end

n : the total number of cuts

ΔX : the multiset of integers representing lengths of each of the $\binom{n}{2}$ fragments produced from a partial digest

One More Partial Digest Example

	0	2	4	7	10
0		2	4	7	10
2			2	5	8
4				3	6
7					3
10					

Representation of $\Delta\mathbf{X} = \{2, 2, 3, 3, 4, 5, 6, 7, 8, 10\}$ as a two dimensional table, with elements of

$$\mathbf{X} = \{0, 2, 4, 7, 10\}$$

along both the top and left side. The elements at (i, j) in the table is the value $|x_i - x_j|$ for $1 \leq i < j \leq n$.

Partial Digest: Multiple Solutions

- It is not always possible to uniquely reconstruct a set X based only on ΔX .

- For example, the set

$$A = \{0, 2, 5\}$$

and

$$(A \oplus 10) = \{10, 12, 15\}$$

both produce $\{2, 3, 5\}$ as their partial digest set.

- The sets $\{0,1,2,5,7,9,12\}$ and $\{0,1,5,7,8,10,12\}$ present a less trivial example of the problem of non-uniqueness. Here both sets digest into:

$$\{1, 1, 2, 2, 2, 3, 3, 4, 4, 5, 5, 5, 6, 7, 7, 7, 8, 9, 10, 11, 12\}$$

Homometric sets

	0	1	2	5	7	9	12
0		1	2	5	7	9	12
1			1	4	6	8	11
2				3	5	7	10
5					2	4	7
7						2	5
9							3
12							

	0	1	5	7	8	10	12
0		1	5	7	8	10	12
1			4	6	7	9	11
5				2	3	5	7
7					1	3	5
8						2	4
10							2
12							

Partial Digest Problem: Formulation

Partial Digest Problem: Given all pairwise distances between points on a line, reconstruct the positions of those points

- Input: The multiset of pairwise distances L , containing $\binom{n}{2}$ integers
- Output: A set X , of n integers, such that $\Delta X = L$

Brute Force Algorithms

- Also known as exhaustive search algorithms; examines every possible solution to find a valid one
- (e.g., list sorting): look at all permutations of elements in the list until finding the sorted version
- Efficient in rare cases; usually impractical

Partial Digest: Brute Force

1. Find the restriction fragment of maximum length M . M is the length of the DNA sequence.
2. For every possible solution, compute the corresponding ΔX
3. If ΔX is equal to the experimental partial digest L , then X is the correct restriction map

BruteForcePDP

1. BruteForcePDP(L, n):
2. $M \leftarrow$ maximum element in L
3. for every set of $n - 2$ integers $0 < x_2 < \dots x_{n-1} < M$
4. $X \leftarrow \{0, x_2, \dots, x_{n-1}, M\}$
5. Form ΔX from X
6. if $\Delta X = L$
7. return X
8. output “no solution”

Efficiency of BruteForcePDP

- The speed of the BruteForcePDP is unfortunately $O(M^{n-2})$ as it must examine all $\binom{M-1}{n-2}$ possible sets of positions.
- One way to improve the algorithm is to limit the values of x_i to only those values which occur in L , as we shall see in the next slide.

AnotherBruteForcePDP

1. AnotherBruteForcePDP(L, n)
2. $M \leftarrow$ maximum element in L
3. for every set of $n - 2$ integers $0 < x_2 < \dots x_{n-1} < M$ from L
4. $X \leftarrow \{0, x_2, \dots, x_{n-1}, M\}$
5. Form ΔX from X
6. if $\Delta X = L$
7. return X
8. output “no solution”

Efficiency of AnotherBruteForcePDP

- It's more efficient, but still slow
- Only $\binom{|L|}{n-1}$ sets examined, but runtime is still exponential: $O(n^{2n-4})$, since $L = n(n-1)/2$
- If $L = \{2, 998, 1000\}$ ($n = 3$, $M = 1000$), BruteForcePDP will be extremely slow, but AnotherBruteForcePDP will be quite fast

Branch and Bound Algorithm for PDP

1. Begin with $X = \{0\}$
2. Remove the largest element in L and place it in X
3. See if the element fits on the right or left side of the restriction map
4. When it fits, find the other lengths it creates and remove those from L
5. Go back to step 1 until L is empty

An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$

$$X = \{ 0 \}$$

An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$
$$X = \{ 0 \}$$

Then n must be 5.

Remove 10 from L and insert it into X . We know this must be
The length of the DNA sequence because it is the largest
fragment.

An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$

$$X = \{ 0, 10 \}$$



An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$

$$X = \{ 0, 10 \}$$

Take 8 from L and make $y = 2$ or 8. But since the two cases are symmetric, we can assume $y = 2$. We find that the distances from y to other elements at X are $\Delta(y, X) = \{8, 2\}$, so we remove $\{8, 2\}$ from L and add 2 to X .



$$\Delta(y, X) = \{|y - x_1|, |y - x_2|, \dots, |y - x_n|\}$$

$$\text{for } X = \{x_1, x_2, \dots, x_n\}$$

An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$

$$X = \{ 0, 2, 10 \}$$



An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$
$$X = \{ 0, 2, 10 \}$$

Take 7 from L and make $y = 7$ or $y = 10 - 7 = 3$. We will explore $y = 7$ first, so $\Delta(y, X) = \{7, 5, 3\}$. Therefore we remove $\{7, 5, 3\}$ from L and add 7 to X .



$$\Delta(y, X) = \{7, 5, 3\} = \{|7 - 0|, |7 - 2|, |7 - 10|\}$$

An Example

$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$

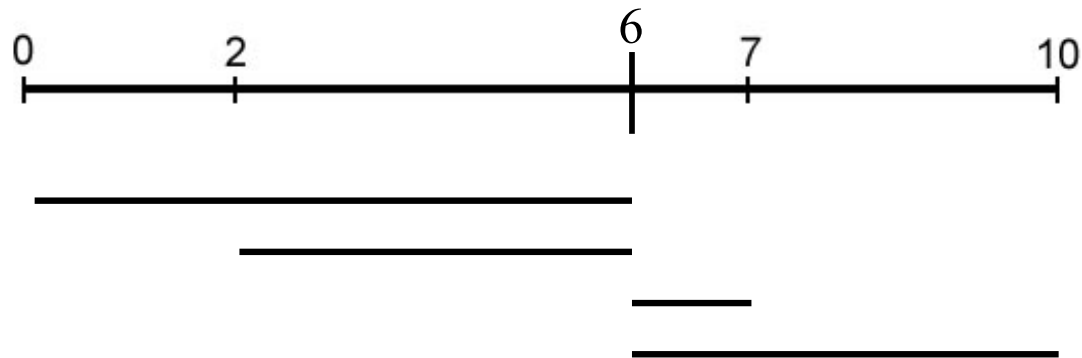
$X = \{ 0, 2, 7, 10 \}$



An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$
$$X = \{ 0, 2, 7, 10 \}$$

Take 6 from L and make $y = 6$. Unfortunately $\Delta(y, X) = \{6, 4, 1, 4\}$, which is not a subset of L . Therefore we won't explore this branch.



An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$
$$X = \{ 0, 2, 7, 10 \}$$

This time make $y = 4$. $\Delta(y, X) = \{4, 2, 3, 6\}$, which is a subset of L so we will explore this branch. We remove $\{4, 2, 3, 6\}$ from L and add 4 to X .



An Example

$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$

$X = \{ 0, 2, 4, 7, 10 \}$



An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$
$$X = \{ 0, 2, 4, 7, 10 \}$$

L is now empty, so we have a solution, which is X .





{2, 2, 3, 3, 4, 5, 6, 7, 8}



{2, 3, 3, 4, 5, 6, 7}



Impossible!



{2, 3, 4, 6}

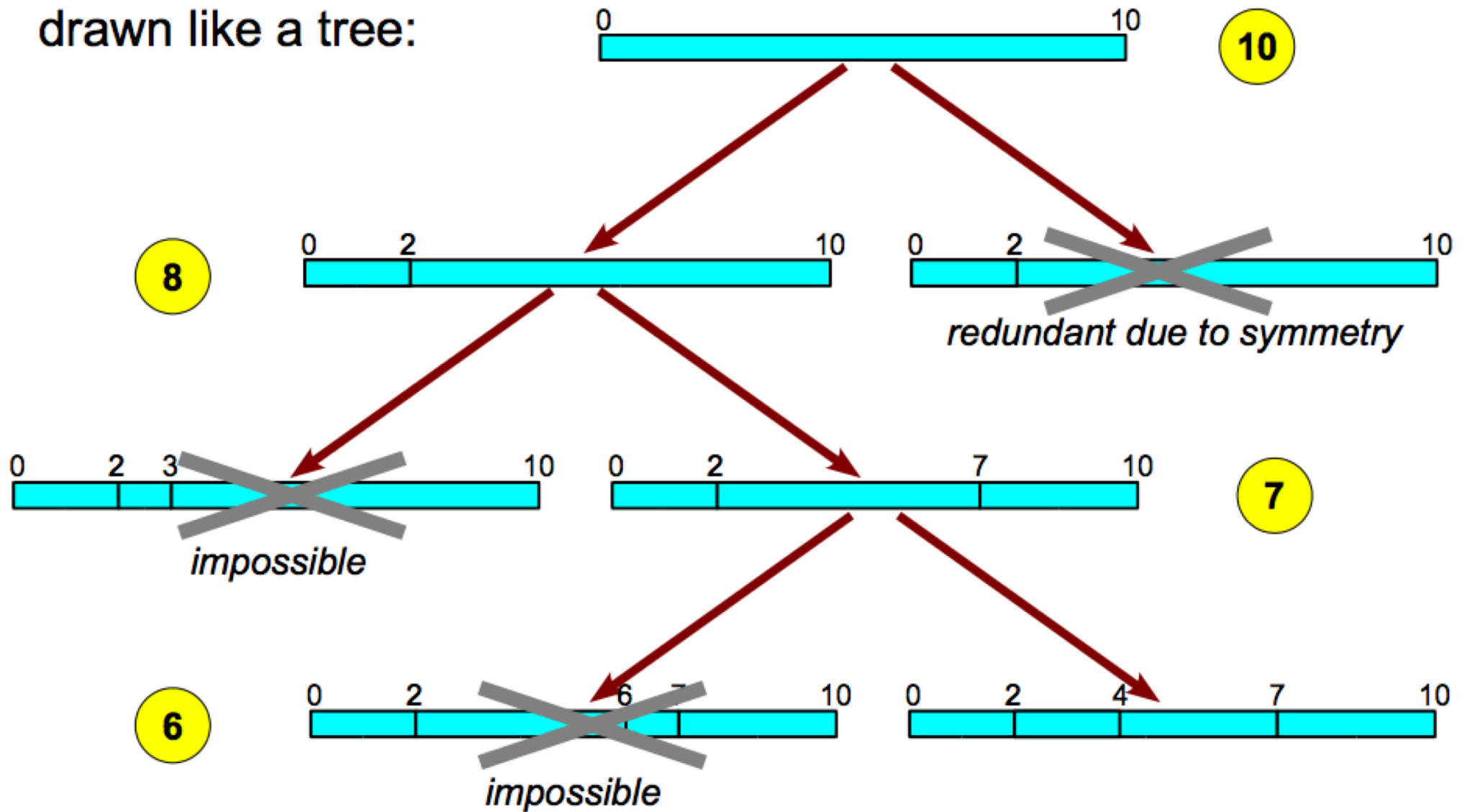


Impossible!



{}

Or, if you prefer,
drawn like a tree:



An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$
$$X = \{ 0, 2, 7, 10 \}$$

To find other solutions, we backtrack.



An Example

$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$

$X = \{ 0, 2, 10 \}$

More backtrack.



An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$
$$X = \{ 0, 2, 10 \}$$

This time we will explore $y = 3$. $\Delta(y, X) = \{3, 1, 7\}$, which is not a subset of L , so we won't explore this branch.



An Example

$$L = \{ 2, 2, 3, 3, 4, 5, 6, 7, 8, 10 \}$$
$$X = \{ 0, 10 \}$$

We backtracked back to the root. Therefore we have found all the solutions.



PartialDigest Algorithm

- First, define $\Delta(y, \mathbf{X})$ as the multiset of all distances between point y and all other points in the set $\mathbf{X}$

PartialDigest(L):

$width \leftarrow$ Maximum element in L

DELETE($width, L$)

$X \leftarrow \{0, width\}$

PLACE(L, X)

PartialDigest Algorithm (cont' d)

1. PLACE(L, X)
2. if L is empty
3. output X
4. return
5. $y \leftarrow$ Maximum element in L
6. If $\Delta(y, X) \subseteq L$
7. Add y to X and remove lengths $\Delta(y, X)$ from L
8. PLACE(L, X)
9. Remove y from X and add lengths $\Delta(y, X)$ to L
10. If $\Delta(width-y, X) \subseteq L$
11. Add $width-y$ to X and remove lengths $\Delta(width-y, X)$
12. PLACE(L, X)
13. Remove $width-y$ from X and add lengths $\Delta(width-y, X)$
14. return

Analyzing PartialDigest Algorithm

- Still exponential in worst case, but is very fast on average
- For n different fragments, if time of PartialDigest is $T(n)$:
 - No branching case: $T(n) < T(n-1) + O(n)$
 - Quadratic
 - Branching case: $T(n) < 2T(n-1) + O(n)$
 - Exponential

Complexity

- DDP is NP-Complete
 - L. Goldstein and M. Waterman. Mapping DNA by stochastic relaxation. *Adv. Appl. Math.*, 1987.
- PDP is polynomial.
 - A. Duarat, Y. Gerard, and M. Nivat. The chords problem. *Theoretical Computer Science*, 282:319-336, 2002.
- EDDP is NP-Hard
 - Ming-Yang Kao, Jared Samet, Wing-Kin Sung, **Proceedings of the 7th Scandinavian Workshop on Algorithm Theory**, p. 383 - 392, 2000.
 - Although this new problem is also NP-hard, it can be solved in linear time in certain theoretically interesting cases.

Regulatory Motifs in DNA Sequences

Outline

- Gene Regulation
- Regulatory Motifs
- Profiles
- The Gold Bug Problem
- The Motif Finding Problem
- Brute Force Motif Finding
- The Median String Problem
- Search Trees
- Branch-and-Bound Motif Search
- Branch-and-Bound Median String Search

Gene Regulation

- Fruit flies, like humans, are susceptible to infections from bacteria and other pathogens.
- One could design the following experiment:
 - Infect flies with a bacterium and measure which genes are switched on as an immune response.
 - From this set of genes, we would like to determine what triggers their activation.
- Many immunity genes in the fruit fly genome have strings that are reminiscent of TCGGGGATTTC, located upstream of the gene's start.

Regulatory Motifs

- These short strings, TCGGGGGATTTC, are examples of ***regulatory motifs***.
- Proteins known as ***transcription factors*** bind to these motifs, encouraging RNA polymerase to transcribe the downstream genes.
- **Motif finding** is the problem of discovering such motifs without any prior knowledge of how the motifs look.

Regulatory Regions

- Every gene contains a regulatory region (RR) typically stretching 100-1000 bp upstream of the transcriptional start site
- Located within the RR are the *Transcription Factor Binding Sites* (TFBS), also known as *motifs*, specific for a given transcription factor

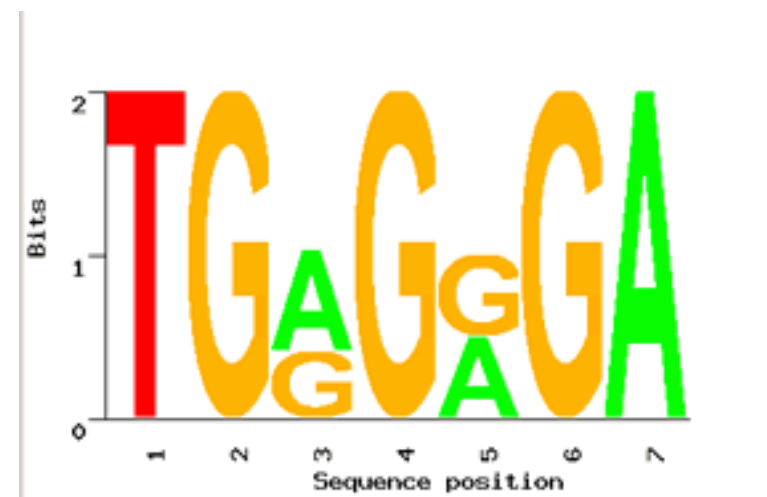
Transcription Factor Binding Sites

- A TFBS can be located anywhere within the Regulatory Region (RR).
- For a single TF to regulate multiple genes, those genes' RRs must contain corresponding TFBS
- TFBS may vary slightly across different regulatory regions since non-essential bases could mutate

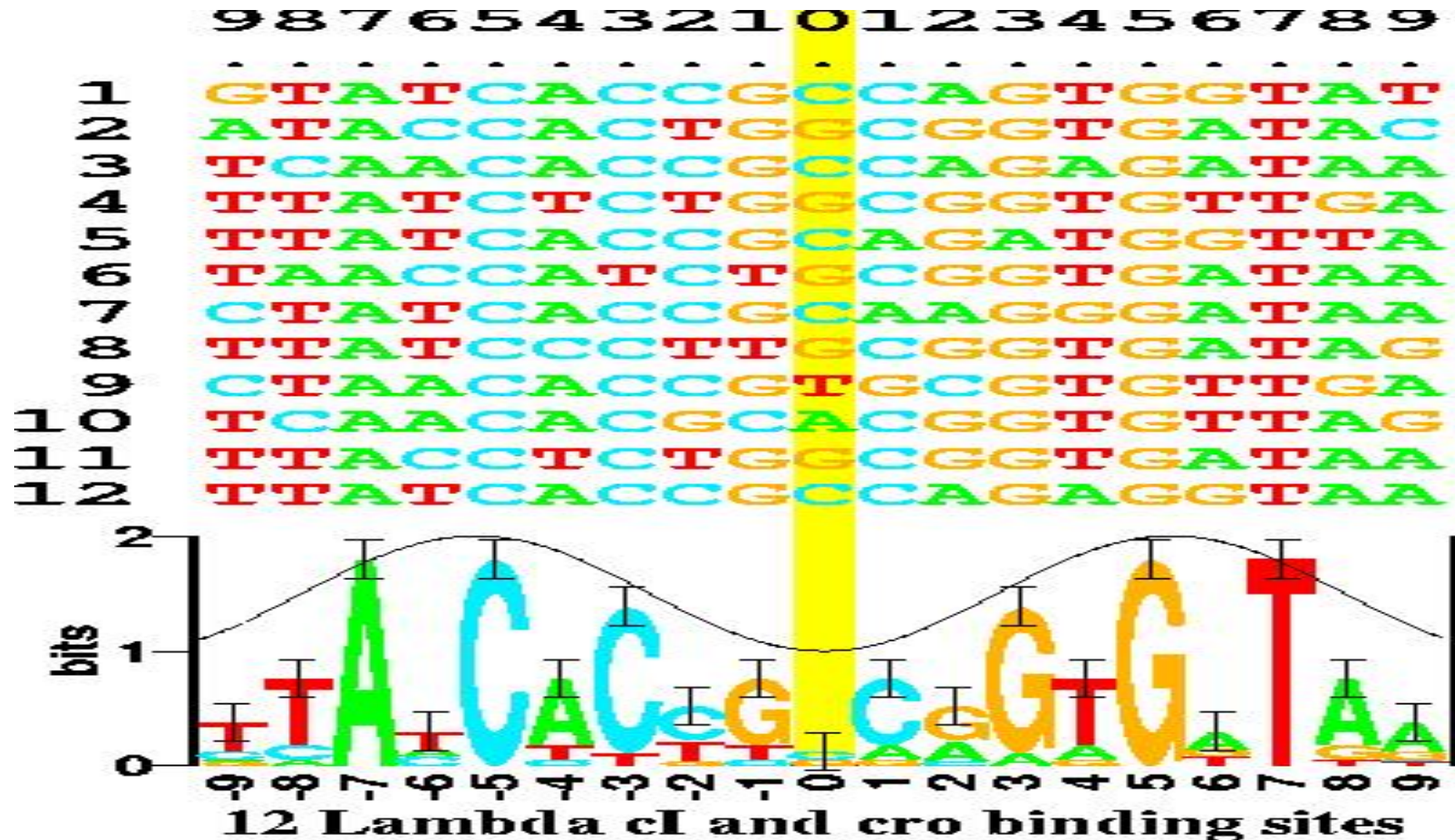
Motif Logo

- Motifs can mutate on non important bases
- The five motifs at top right have mutations in position 3 and 5
- Representations called *motif logos* illustrate the conserved regions of a motif

TGGGGGA
TGAGAGA
TGGGGGA
TGAGAGA
TGAGGGA



Motif Logos: An Example



Motifs and Transcriptional Start Sites



Identifying Motifs

- Genes are turned on or off by regulatory proteins
- These proteins bind to upstream regulatory regions of genes to either attract or block an RNA polymerase
- Regulatory protein X binds to a unique DNA sequence in the region, called a motif
- So finding X' s motif in multiple genes' regulatory regions suggests a regulatory relationship amongst those genes

Identifying Motifs: Complications

- We do not know the motif sequence
- We do not know where it is located relative to the genes start
- Motifs can differ slightly from one gene to the next
- How to discern it from “random” motifs?

Random Sample

atgaccgggatactgataccgtatttggcctaggcgtacacattagataaacgtatgaagtacgttagactcggcgccgcccg
accctatTTTTTgagcagatttagtgacctggaaaaaaaaatttgagtacaaaacttttccgaatactgggcataaggtaca
tgagtatccctgggatgacttttgggaacactatagtgtctctcccgatttttgaatatgtaggatcattcgccaggggtccga
gctgagaattggatgaccttgtaagtgttttccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
tcccttttgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaatggcccacttagtccacttatag
gtcaatcatgttcttgtgaatggattttttaactgagggcatagaccgcttggcgcacccaaattcagtggtggcgagcgcaa
cggttttggcccttgtagaggccccgtactgatggaaactttcaattatgagagagctaattctatcgcggtgcgtgttcat
aacttgagttggtttcgaaaatgctctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
ttggcccatgggctaaaagcccaacttgacaaatggaagatagaatccttgcatTTTcaacgtatgccgaaccgaaaggggaag
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttctgggtactgatagca

Implanting Motif AAAAAAAGGGGGGGG

atgaccgggatactgatAAAAAAAGGGGGGGGggcggtacacattagataaacgtatgaagtacgttagactcggcgccgccg
accctatTTTTTtgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaataAAAAAAAGGGGGGGGa
tgagtatccctgggatgacttAAAAAAAGGGGGGGGtgctctcccgattTTTgaatatgtaggatcattcgccagggtccga
gctgagaattggatgAAAAAAAGGGGGGGGtcacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
tccTTTTtgcggtaatgtgccgggaggctggttacgtaggggaagccctaacggacttaatAAAAAAAGGGGGGGGcttatag
gtcaatcatgttcttgtgaatggatttAAAAAAAGGGGGGGGgaccgcttggcgcacccaaattcagtggtggcgagcgcaa
cggTTTTtgcccttggttagaggcccccgTAAAAAAAGGGGGGGGcaattatgagagagctaattctatcgcggtgcgtgttcat
aacttgagttAAAAAAAGGGGGGGGctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
ttggccattggctaaaagcccaacttgacaaatggaagatagaatccttgcatAAAAAAAGGGGGGGGaccgaaaggggaag
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttAAAAAAAGGGGGGGGa

Where is the Implanted Motif?

atgaccgggatactgataaaaaaaagggggggggcggtacacattagataaacgtatgaagtacgttagactcggcgccgccg
accctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaataaaaaaaaggggggga
tgagtatccctgggatgacttaaaaaaaaggggggggtgctctcccgattTTTgaatatgtaggatcattcgccagggtccga
gctgagaattggatgaaaaaaaggggggggtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
tccctTTTgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaataaaaaaaagggggggcTTtatag
gtcaatcatgttcttTgtgaatggatttaaaaaaaaggggggggaccgcttggcgcacccaaattcagtgTgggagcgcaa
cggTTTTggccctTgttagaggccccgtaaaaaaaaggggggggcaattatgagagagctaattctatcgcgTgcgtgtTcat
aacttgagTtaaaaaaaagggggggctggggcacatacaagaggagtcttcttatcagTtaatgctgtatgacactatgta
ttggcccatTggctaaaagcccaacttgacaaatggaagatagaatccttgcataaaaaaaagggggggaccgaaagggaag
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttaaaaaaaaggggggga

Implanting Motif AAAAAAAGGGGGGGG with Four Mutations

atgaccgggatactgatAgAAgAAAGGttGGGggcggtacacattagataaacgtatgaagtacgttagactcggcgccgccg
accctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaacttttccgaataCAAtAAACGGcGGGa
tgagtatccctgggatgacttAAAAtAAtGGaGtGGtgctctcccgatttttgaatatgtaggatcattcgccagggtccga
gctgagaattggatgCAAAAAAGGGAttGtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
tcccttttgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaAtAAtAAAGGaaGGGcttatag
gtcaatcatgttcttgtgaatggatttAAcAAAtAAGGGctGGgaccgcttggcgcacccaaattcagtggtggcgagcgcaa
cggTTTTTggcccttggttagaggccccgtAtAAACAAAGGaGGGccaattatgagagagctaattctatcgcggtgcgtgttcat
aacttgagttAAAAAAtAGGGaGccctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
ttggccatttggtctaaaagcccaacttgacaaatggaagatagaatccttgcatActAAAAAGGaGcGGaccgaaaggggaag
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttActAAAAAGGaGcGGa

Where is the Motif???

atgaccgggatactgatagaagaaagggttgggggcggtacacattagataaacgtatgaagtacgttagactcggcgccgccg
accctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaacttttccgaatacaataaaacggcgggga
tgagtatccctgggatgacttaaaataatggagtggtgctctcccgatttttgaatatgtaggatcattcgccaggggtccga
gctgagaattggatgcaaaaaagggttgtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
tcccttttgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaataataaaggaagggcttatag
gtcaatcatgttcttgtgaatggatttaacaataagggctgggaccgcttggcgcacccaaattcagtggtggcgagcgcaa
cggttttggcccttgtagaggccccgtataaacaaggagggccaattatgagagagctaattctatcgcggtgcgtgttcat
aacttgagttaaaaaatagggagccctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
ttggcccatggctaaaagcccaacttgacaaatggaagatagaatccttgcatactaaaaaggagcggaaccgaaaggggaag
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttactaaaaaggagcgga

Challenge Problem

- Find a motif in a sample of
 - 20 “random” sequences (e.g. 600 nt long)
 - each sequence containing an implanted pattern of length 15,
 - each pattern appearing with 4 mismatches as (15,4)-motif.
-

A Motif Finding Analogy



- The Motif Finding Problem is similar to The Gold Bug Problem posed by Edgar Allan Poe two centuries ago

The Gold Bug Problem

- By Edgar Allan Poe (1809 – 1849)

- Given a secret message:

```
53++!305) ) 6*;4826) 4+.) 4+);806*;48!8`60)) 85;]8*:+*8!83(88)5*!;  
46(;88*96*?;8)*+(;485);5*!2:*+(;4956*2(5*-4)8`8*; 4069285);) 6  
!8)4++;1(+9;48081;8:8+1;48!85;4)485!528806*81(+9;48;(88;4(+?3  
4;48)4+;161;:188;+?;
```

- Decipher the message encrypted in the fragment

Hints for The Gold Bug Problem

- Additional hints:
 - The encrypted message is in English
 - Each symbol correspond to one letter in the English alphabet
 - No punctuation marks are encoded

The Gold Bug Problem: Symbol Counts

- Naive approach in solving the problem:
 - Count the frequency of each symbol in the encrypted message
 - Find the frequency of each letter in the alphabet in the English language
 - Compare the frequencies of the previous steps, try to find a correlation and map the symbols to a letter in the alphabet

Symbol Frequencies in the Gold Bug Message

- Gold Bug Message:**

Symbol	8	;	4	)	+	*	5	6	(	!	1	0	2	9	3	:	?	`	-	]	.
Frequency	34	25	19	16	15	14	12	11	9	8	7	6	5	5	4	4	3	2	1	1	1

- English Language:**

e t a o i n s r h l d c u m f p g w y b v k x j q z

Most frequent

Least frequent



The Gold Bug Message Decoding: First Attempt

- By simply mapping the most frequent symbols to the most frequent letters of the alphabet:

```
sfiilfcsoorntaeuroaikoaiotecrntaeleyrcooestvenpinelefheeosnlt  
arhteenmrnwteonihtaesotsnlupnihtamsrnuhsnbaoeyentacrmuesotorl  
eoaiitdhimtaecedtepeidtaelestaoaeslsueecrnedhimtaetheetahiwfa  
taeoaitdrdtpdeetiwt
```

- The result does not make sense

The Gold Bug Problem: l-tuple count

- A better approach:
 - Examine frequencies of tuples, combinations of 2 symbols, 3 symbols, etc.
 - “The” is the most frequent 3-tuple in English and “;48” is the most frequent 3-tuple in the encrypted text
 - Make inferences of unknown symbols by examining neighboring decrypted letters

The Gold Bug Problem: the ;48 clue

- Mapping “the” to “;48” and substituting all occurrences of the symbols:

```
53++!305) ) 6*the26)h+.)h+)te06*the!e`60))e5t]e*:+*e!e3(ee)5*!t
h6(tee*96*?te)*+(the5)t5*!2:*+(th956*2(5*h)e`e*th0692e5)t)6!e
)h++t1(+9the0e1te:e+1the!e5th)he5!52ee06*e1(+9thet(eeth(+?3ht
he)h+t161t:1eet+?t
```

The Gold Bug Message Decoding: Second Attempt

- Make inferences:

```
53++!305) ) 6*the26)h+.)h+)te06*the!e`60))e5t]e*:+*e!e3(ee)5*!t
h6(tee*96*?te)*+(the5)t5*!2:*+(th956*2(5*h)e`e*th0692e5)t)6!e
)h++t1(+9the0e1te:e+1the!e5th)he5!52ee06*e1(+9thet(eeth(+?3ht
he)h+t161t:1eet+?t
```

- “~~thet~~(~~ee~~” most likely means “the tree”
 - Infer “(“ = “r”
- “th(+?h” becomes “thr+?h”
 - Can we guess “+” and “?”?

The Gold Bug Problem: The Solution

- After figuring out all the mappings, the final message is:

AGOODGLASSINTHEBISHOPSHOSTELINTHEDEVILSSEATWENYONEDEGRE
ESANDTHIRTEENMINUTESNORTHEASTANDBYNORTHMAINBRANCHSEVENT
HLIMBEASTSIDESHOOTFROMTHELEFTEYE OF THE DEATHS HEAD A BEELINE
FROM THE TREE THROUGH THE SHOT FIFTY FEET OUT

The Solution (cont' d)

- Punctuation is important:

A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEA,
TWENY ONE DEGREES AND THIRTEEN MINUTES NORTHEAST AND BY NORTH,
MAIN BRANCH SEVENTH LIMB, EAST SIDE, SHOOT FROM THE LEFT EYE OF
THE DEATH'S HEAD A BEE LINE FROM THE TREE THROUGH THE SHOT,
FIFTY FEET OUT.

Solving the Gold Bug Problem

- Prerequisites to solve the problem:
 - Need to know the relative frequencies of single letters, and combinations of two and/or three letters in English
 - Knowledge of all the words in the English dictionary is highly desired for us to make accurate inferences

Motif Finding and The Gold Bug Problem: Similarities

- **Motif Finding:**
 - Nucleotides in motifs encode for a message in the “genetic” language
- **Gold Bug Problem:**
 - Symbols encode for a message in English

Similarities (cont' d)

- **Motif Finding:**
 - In order to solve the problem, we record the frequencies of patterns in the nucleotide sequences
- **Gold Bug Problem:**
 - The approach to solve the problem is to find out the frequencies of the patterns

Similarities (cont' d)

- **Motif Finding:**
 - Knowledge of established motifs reduces the complexity of the problem
- **Gold Bug Problem:**
 - Knowledge of the words in the dictionary is highly desirable

Motif Finding and The Gold Bug Problem: Differences

- **Motif Finding** is harder than **Gold Bug problem**:
 - We don't have the complete dictionary of motifs
 - The “genetic” language does not have a standard “grammar”
 - Only a small fraction of nucleotide sequences encode for motifs; the size of data is enormous
 - Human intelligence is focused on creating good algorithms for the problem rather than on making inferences and guesses on the possible motifs in the sequence

The Motif Finding Problem

- Given a random sample of DNA sequences:

```
cctgatagacgctatctggctatccacgtacgtaggtcctctgtgcgaatctatgcgtttccaacat  
agtactgggtgtacatttgatacgtacgtacaccggcaacctgaaacaaacgctcagaaccagaagtgc  
aaacgtacgtgcaccctctttcttcgtggctctggccaacgagggctgatgtataagacgaaaatttt  
agcctccgatgtaagtcatagctgtaactattacctgccaccctattacatcttacgtacgtataca  
ctgttatacaacgcgctcatggcgggggtatgcgttttggctcgtcgtacgctcgatcgttaacgtacgtc
```

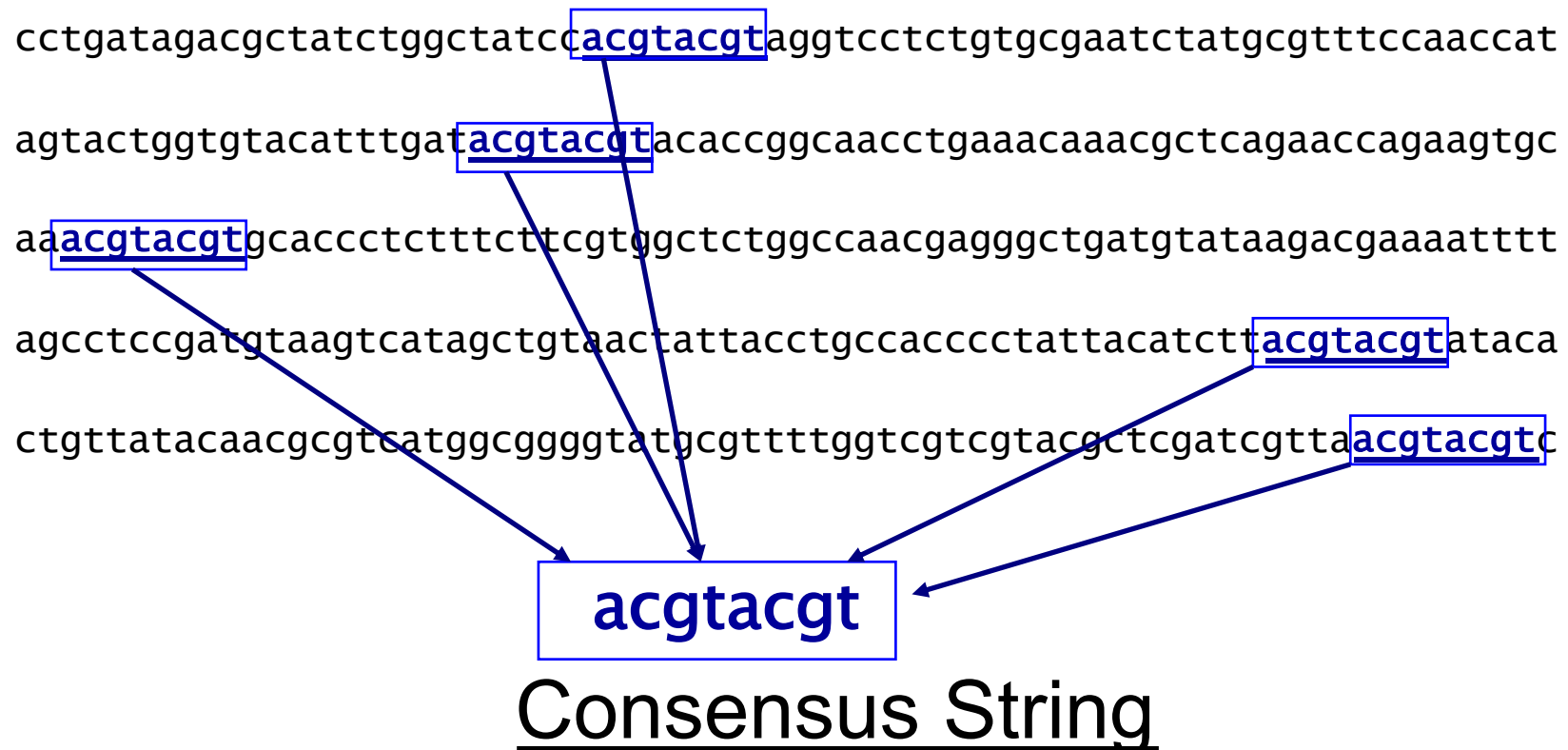
- Find the pattern that is implanted in each of the individual arrays, namely, the motif

The Motif Finding Problem (cont' d)

- Additional information:
 - The hidden sequence is of length 8

The Motif Finding Problem (cont' d)

- The patterns revealed:



The Motif Finding Problem (cont' d)

- We could simply count the number of times each 8-mer occurs in the sample.
- From above example, since there are only $5 \cdot (69) = 345$ nucleotides in the sample, it is unlikely that any 8-mer other than the implanted pattern appears more than twice.

$$\text{Pr}(8\text{-mer appears in the sample}) \approx 345/4^8 < 0.006$$

The Motif Finding Problem (cont' d)

- The pattern is not exactly the same in each array because random point mutations may occur in the sequences
- The patterns with 2 point mutations:

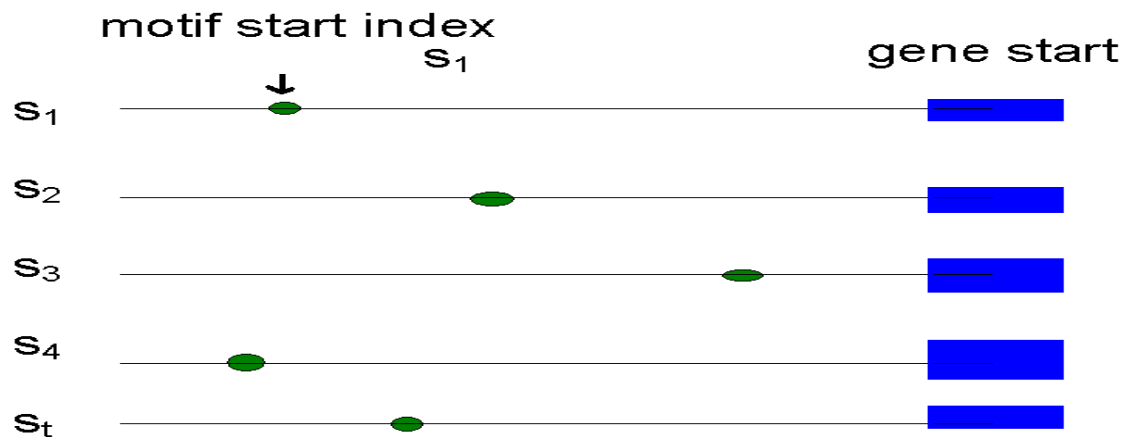
```
cctgatagacgctatctggctatccaGgtacTtaggtcctctgtgCGaatctatgCGtttccaacat
agtactggtgtacatttgatCcAtacgtacaccggcaacctgaaacaaacgctcagaaccagaagtgc
aaacgtTAgtgcaccctctttcttcgtggctctggccaacgagggctgatgtataagacgaaaatttt
agcctccgatgtaagtcatagctgtaactattacctgccacccctattacatcttacgtCcAtataca
ctgttatacaacgCGtcatggcgggggtatgCGttttggtcgTCgtacgctcgatcgTTaCcgtacgGc
```

The Motif Finding Problem (cont' d)

- The problem:
 - Can we still find the motif, now that we have 2 mutations?
 - What is the consensus sequence?

Defining Motifs

- To define a motif, let's say we know where the motif starts in the sequence
- The motif start positions in their sequences can be represented as $\mathbf{s} = (s_1, s_2, s_3, \dots, s_t)$



Motifs: Profiles and Consensus

Alignment

```

a G g t a c T t
C c A t a c g t
a c g t T A g t
a c g t C c A t
C c g t a c g G
    
```

- Line up the patterns by their start indexes

$$\mathbf{s} = (s_1, s_2, \dots, s_t)$$

Profile

A	3	0	1	0	3	1	1	0
C	2	4	0	0	1	4	0	0
G	0	1	4	0	0	0	3	1
T	0	0	0	5	1	0	1	4

- Construct matrix profile with frequencies of each nucleotide in columns

Consensus

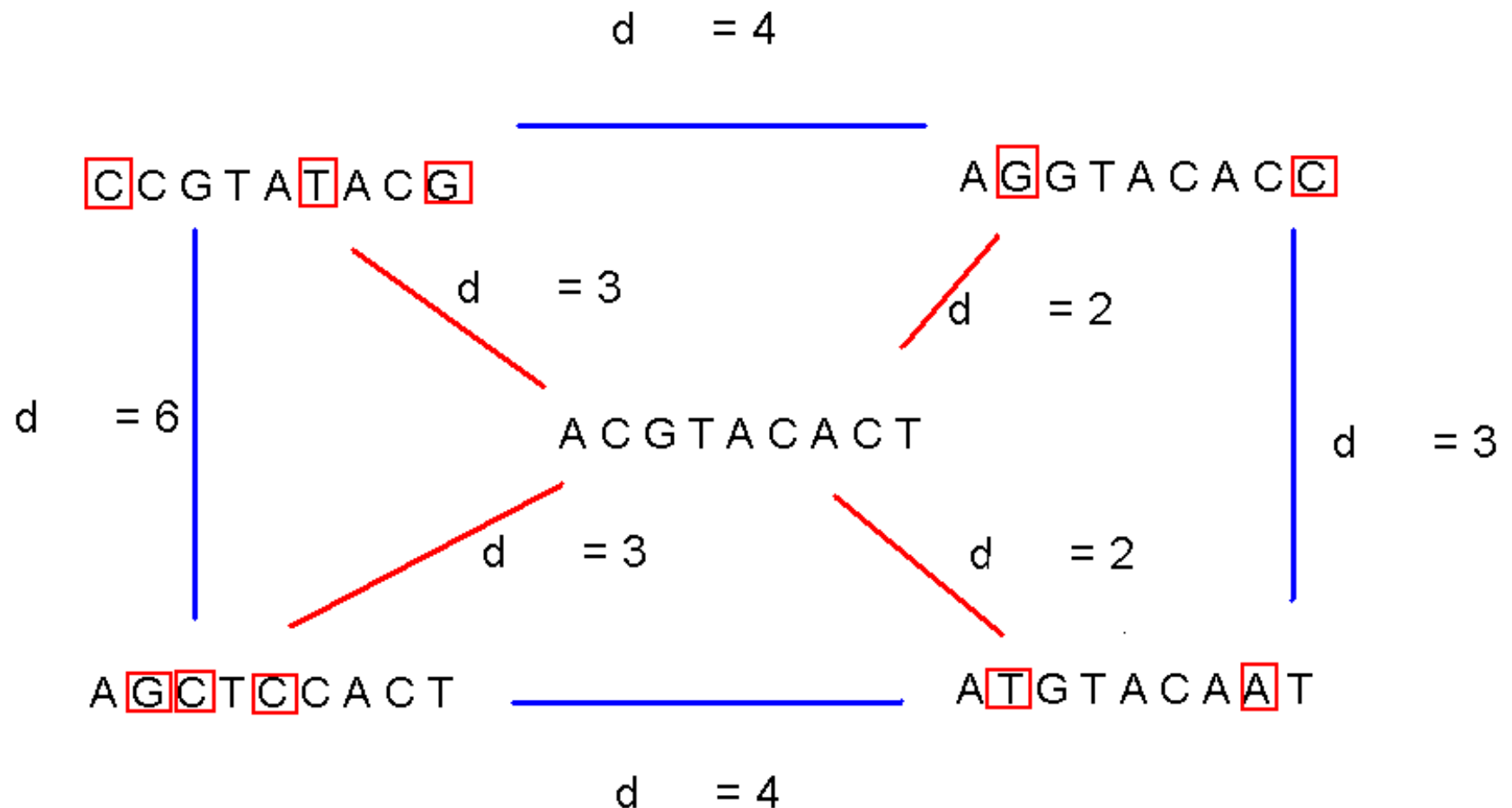
A C G T A C G T

- Consensus nucleotide in each position has the highest score in column

Consensus

- Consensus sequences help in finding motifs
- Think of consensus as an “ancestor” motif, from which mutated motifs emerged
- The *distance* between a real motif and the consensus sequence is generally less than that for two real motifs

Consensus (cont' d)



Evaluating Motifs

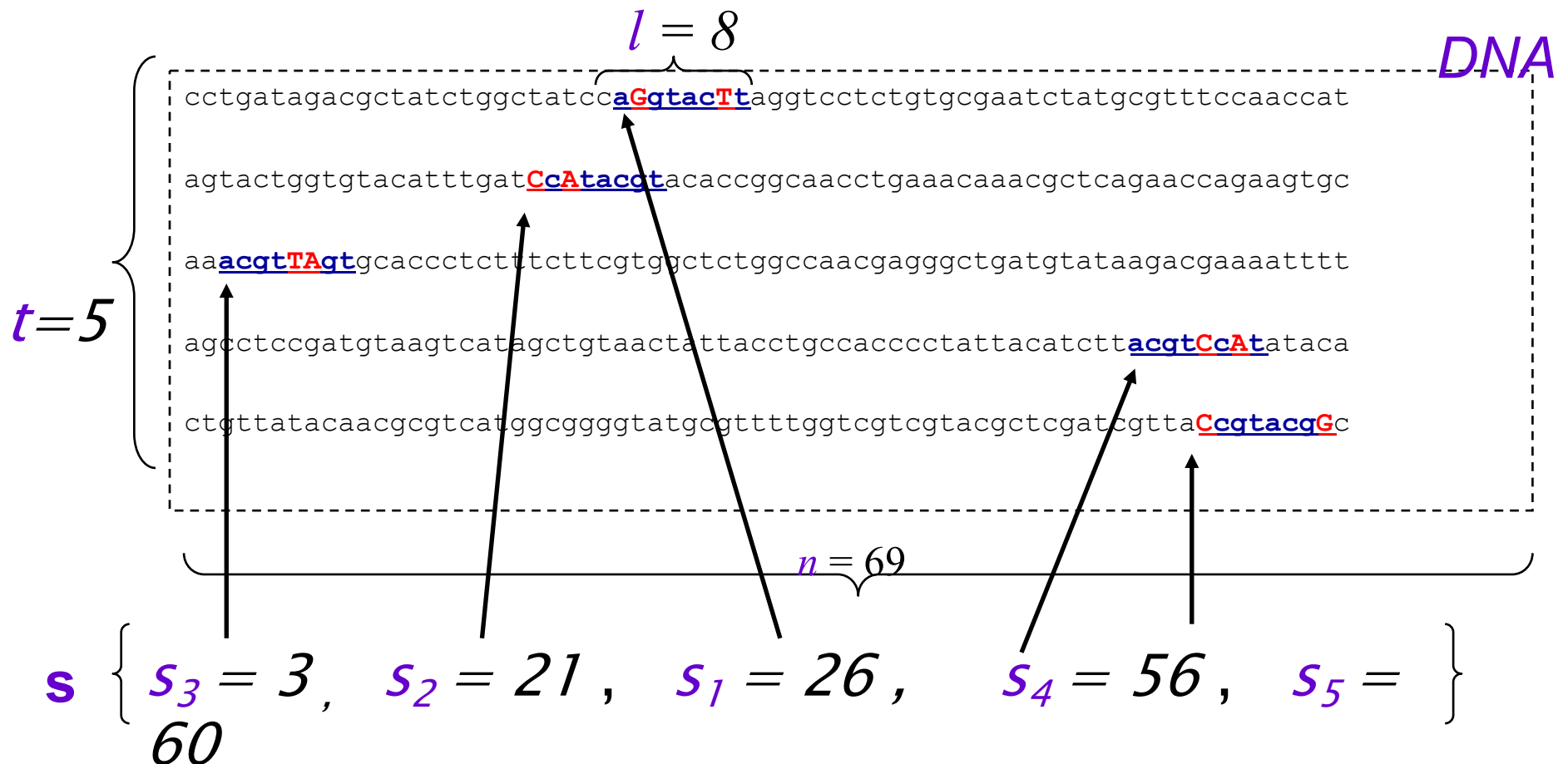
- We found the consensus sequence, but how “good” is this consensus?
- Need to introduce a scoring function
- However, we need to introduce some parameters...

Defining Some Terms

- t - number of sample DNA sequences
- n - length of each DNA sequence
- **DNA** - sample of DNA sequences ($t \times n$ array)
- ℓ - length of the motif (ℓ mer)
- s_i - starting position of an ℓ mer in sequence i
- $\mathbf{s} = (s_1, s_2, \dots, s_t)$ - array of motif's starting positions
- $\mathbf{P}(\mathbf{s})$ - denotes the profile matrix corresponding to starting positions $\mathbf{s}$
- $M_{\mathbf{P}(\mathbf{s})}(j)$ - denotes the largest count in column j of $\mathbf{P}(\mathbf{s})$

Parameters

- In our sample sequence:



Scoring Function

- Given $\mathbf{s} = (s_1, \dots, s_t)$ and **DNA**:

$$\text{Score}(\mathbf{s}, \mathbf{DNA}) = \sum_{j=1}^l M_{P(S)}(j)$$

l								t
a	G	g	t	a	c	T	t	
C	c	A	t	a	c	g	t	
a	c	g	t	T	A	g	t	
a	c	g	t	C	c	A	t	
C	c	g	t	a	c	g	G	

A	3	0	1	0	3	1	1	0
C	2	4	0	0	1	4	0	0
G	0	1	4	0	0	0	3	1
T	0	0	0	5	1	0	1	4

Consensus **a c g t a c g t**

Score **3+4+4+5+3+4+3+4=30**

The Motif Finding Problem

- If starting positions $\mathbf{s}=(s_1, s_2, \dots, s_t)$ are given, the problem is easy even with mutations in the sequences because we can simply construct the profile to find the motif (consensus)
- But... the starting positions $\mathbf{s}$ are usually not given. How can we align the patterns and compute the “best” profile matrix?

The Motif Finding Problem: Formulation

The Motif Finding Problem: Given a set of DNA sequences, find a set of ℓ mers, one from each sequence, that maximizes the consensus score

- Input: A $t \times n$ matrix of *DNA*, and ℓ the length of the pattern to find
- Output: An array of t starting positions $\mathbf{s} = (s_1, s_2, \dots, s_t)$ maximizing $\text{Score}(\mathbf{s}, \text{DNA})$

The Motif Finding Problem: Brute Force Solution

- Compute the scores for each possible combination of starting positions **s**
- The best score will determine the best profile and the consensus pattern in **DNA**
- The goal is to maximize $\text{Score}(\mathbf{s}, \mathbf{DNA})$ by varying the starting positions **s_i**, where:

$$\begin{aligned} s_i &= [1, \dots, n - l + 1] \\ i &= [1, \dots, t] \end{aligned}$$

Brute Force Approach: Running Time

- Varying $(n - \ell + 1)$ positions in each of t sequences, we're looking at $(n - \ell + 1)^t$ sets of starting positions
- For each set of starting positions, the scoring function makes ℓ operations, so complexity is $\ell(n - \ell + 1)^t = O(\ell n^t)$

Pseudocode for Brute Force Motif Search

1. $\text{BruteForceMotifSearch}(DNA, t, n, \emptyset)$
2. $bestScore \leftarrow 0$
3. for each $s = (s_1, s_2, \dots, s_t)$ from $(1, 1, \dots, 1)$
 to $(n - A + 1, \dots, n - A + 1)$
4. if ($Score(s, DNA) > bestScore$)
5. $bestScore \leftarrow score(s, DNA)$
6. $bestMotif \leftarrow (s_1, s_2, \dots, s_t)$
7. return $bestMotif$

Running Time of BruteForceMotifSearch

- That means that for $t = 8$, $n = 1000$, $\ell = 10$
 - Must perform $7.322\text{E}+25$ computations
 - Assuming each computation takes a cycle on a 3 GHz CPU, it would take 7.33 billion years to search all the possibilities
- This algorithm is not practical
- Lets explore some ways to speed it up

Motif Finding Problem: New Formulation

- The motif finding problem is a maximization problem
- Can we convert this maximization problem into a minimization problem?
- The ***Median String Problem***

The Median String Problem

- Given a set of t DNA sequences find a pattern that appear in all t sequences with the minimum number of mutations
- This pattern will be the motif

Hamming Distance

- Hamming distance:
 - $d_H(\mathbf{v}, \mathbf{w})$ is the number of nucleotide pairs that do not match when $\mathbf{v}$ and $\mathbf{w}$ are aligned. For example:

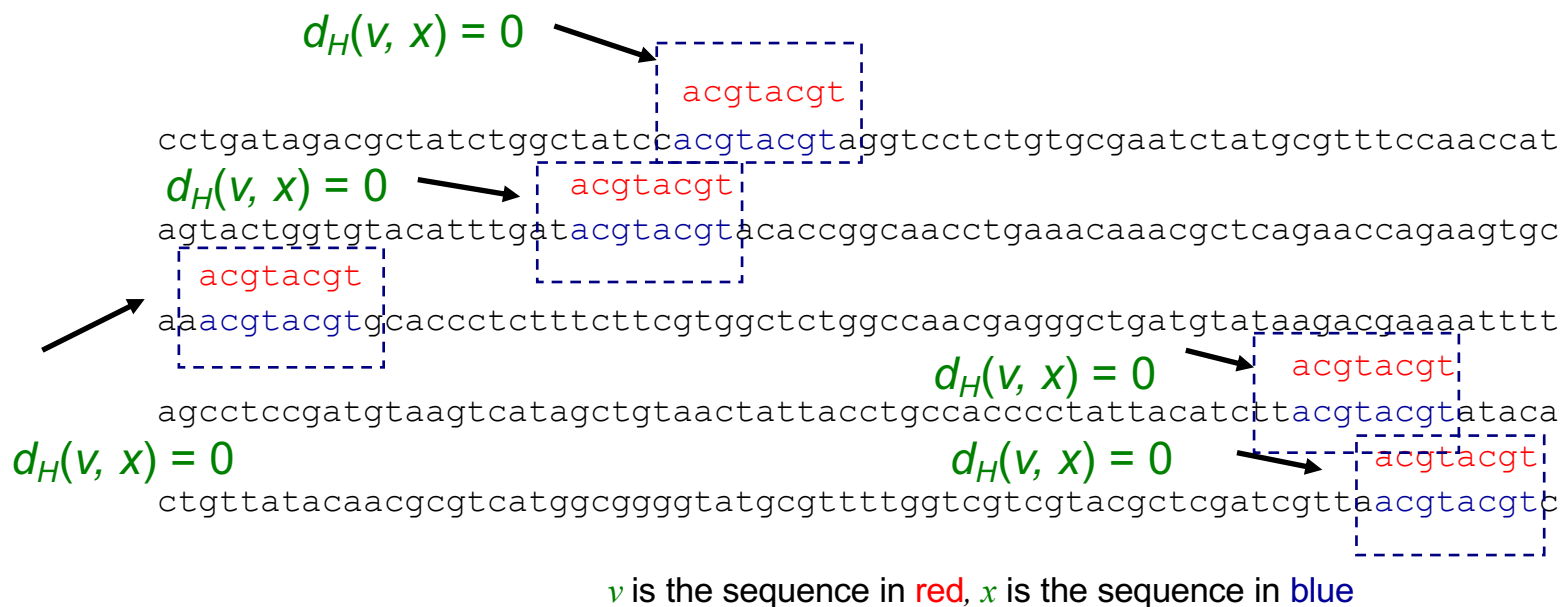
$$d_H(\text{AAAAAA}, \text{ACAAAC}) = 2$$

Total Distance

- Given an ℓ -mer $\mathbf{v}$, $d_H(\mathbf{v}, \mathbf{s})$ denotes the total hamming distance between $\mathbf{v}$ and the ℓ -mers starting at positions $\mathbf{s}$ ($1 \leq s_i \leq n - \ell + 1$)
- $TotalDistance(\mathbf{v}, \mathbf{DNA}) = \min_{\mathbf{s}} d_H(\mathbf{v}, \mathbf{s})$, where $\mathbf{s}$ is the set of starting positions $s_1, s_2, \dots, s_t$

Total Distance: An Example

- Example 1, given $v = \text{"acgtacgt"}$ and s



- $TotalDistance(v, DNA) = 0$

Total Distance: Another Example

- Example 2, given $v = \text{"acgtacgt"}$ and s

$$d_H(v, x) = 1$$

cctgata~~g~~acgctatctg~~g~~cctatc~~c~~acgtac~~A~~taggtcctctgtg~~c~~gaatctatgcg~~t~~ttccaac~~c~~at

$$d_H(v, x) = 0$$

agtactggtgtacatttga~~t~~acgtacgtacaccggcaac~~c~~tga~~a~~acaaacg~~t~~cagaaccagaag~~t~~gc

$$d_H(v, x) = 2$$

aa~~a~~acgt~~C~~cg~~t~~gcaccctctttctt~~c~~gtggctctggccaacgagggctgatgtataagacgaaaat~~t~~tt

agcctccgatgtaagtcata~~g~~ctgtaactattac~~c~~tgccacccctattacat~~c~~ttacgtacgtata~~c~~a

$$d_H(v, x) = 0$$

ctgttatacaacgcgtcatggcg~~g~~gggtatgcg~~t~~tttgg~~t~~cgtcgtacgctcgatcgt~~t~~aacgtacgt~~c~~

$$d_H(v, x) = 1$$

v is the sequence in red, x is the sequence in blue

- $TotalDistance(v, DNA) = 1 + 2 + 1 = 4$

The Median String Problem: Formulation

The Median String Problem:

- Given a set of DNA sequences, find a median string
- Input: A $t \times n$ matrix DNA , and ℓ the length of the pattern to find
- Output: A string $\mathbf{v}$ of ℓ nucleotides that minimizes $TotalDistance(\mathbf{v}, \mathbf{DNA})$ over all strings of that length

Median String Search Algorithm

1. MedianStringSearch (*DNA*, *t*, *n*, *ℓ*)
2. ***bestWord* $\leftarrow$ AAAAAA...AAAA**
3. ***bestDistance* $\leftarrow \infty$**
4. for each *ℓ*-mer ***v*** from AAA...A to TTT...T
 if *TotalDist*(***v***, *DNA*) < ***bestDistance***
5. ***bestDistance* $\leftarrow$ *TotalDist*(***v***, *DNA*)**
6. ***bestWord* $\leftarrow$ ***v*****
7. **return *bestWord***

Motif Finding Problem == Median String Problem

- The *Motif Finding* and *Median String* problems are computationally equivalent
- Proof:
Need to show that minimizing *TotalDistance* is equivalent to maximizing *Score*

We are looking for the same thing

Alignment

a	G	g	t	a	c	T	t
C	c	A	t	a	c	g	t
a	c	g	t	T	A	g	t
a	c	g	t	C	c	A	t
C	c	g	t	a	c	g	G

Profile

A	3	0	1	0	3	1	1	0
C	2	4	0	0	1	4	0	0
G	0	1	4	0	0	0	3	1
T	0	0	0	5	1	0	1	4

Consensus a c g t a c g t

Score 3+4+4+5+3+4+3+4

TotalDistance 2+1+1+0+2+1+2+1

Sum 5 5 5 5 5 5 5 5

- At any column i
 $Score_i + TotalDistance_i = t$
- Because there are l columns
 $Score + TotalDistance = l * t$
- Rearranging:
 $Score = l * t - TotalDistance$
- $l * t$ is constant the minimization of the right side is equivalent to the maximization of the left side

The Motif Finding Problem vs. The Median String Problem

- Why bother reformulating the motif finding problem into the median string problem?
 - The Motif Finding Problem needs to examine all the combinations for **s**. That is $(n - \ell + 1)^t$ combinations!!!
 - The Median String Problem needs to examine all 4^ℓ combinations for **v**. This number is relatively smaller

Motif Finding: Improving the Running Time

Recall the Brute Force Search algorithm:

1. BruteForceMotifSearch($DNA, t, n, \emptyset$)
2. $bestScore \leftarrow 0$
3. for each $s(s_1, s_2, \dots, s_t)$ from $(1, 1, \dots, 1)$ to $(n-t+1, \dots, n-t+1)$
4. if ($Score(s, DNA) > bestScore$)
5. $bestScore \leftarrow score(s, DNA)$
6. $bestMotif \leftarrow (s_1, s_2, \dots, s_t)$
7. return $bestMotif$

Structuring the Search

- How can we perform the line

for each $s(s_1, s_2, \dots, s_t)$ from $(1, 1, \dots, 1)$ to $(n-A+1, \dots, n-A+1)$?

- We need a method for efficiently structuring and navigating the many possible motifs

Alternative Representation of the Search Space

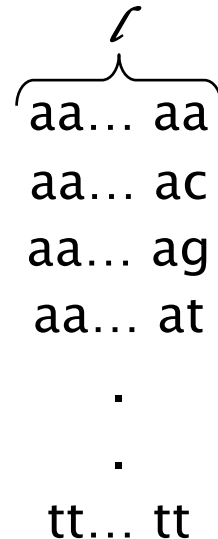
- Let **A** = 1, **C** = 2, **G** = 3, **T** = 4
- Then the sequences become:

↙
┌───┐
11...11
11...12
11...13
11...14
.
.
44...44

- Notice that the sequences above is the set on numbers as if we were counting on base 4 without using 0 as a digit

Structuring the Search

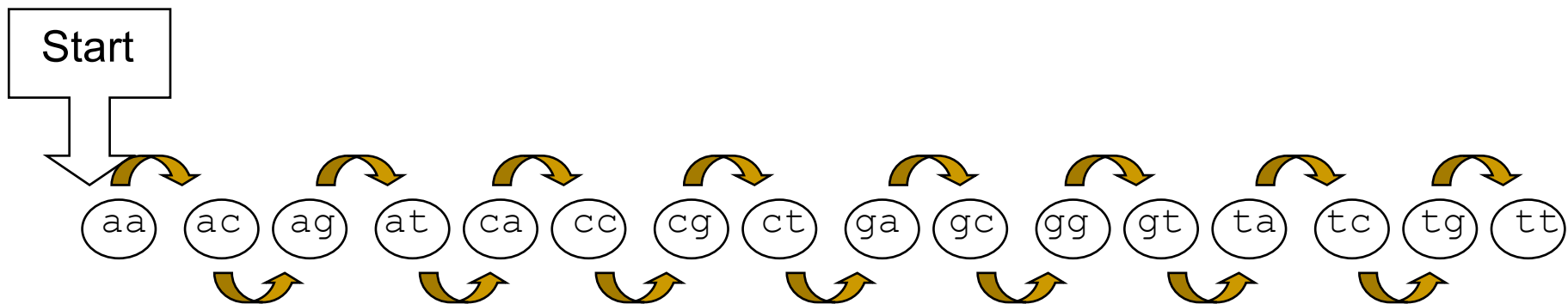
- Let's consider the Median String Problem:
 - For the Median String Problem we need to consider all 4^L possible L mers:


aa... aa
aa... ac
aa... ag
aa... at
.
.
tt... tt

- How to organize this search?

Linked List

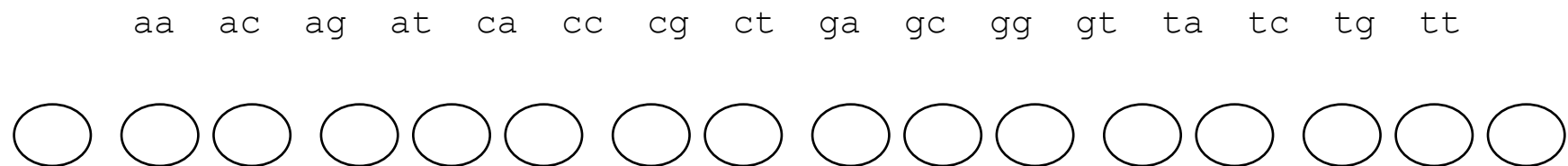
- Suppose $\ell = 2$



- Need to visit all the predecessors of a sequence before visiting the sequence itself

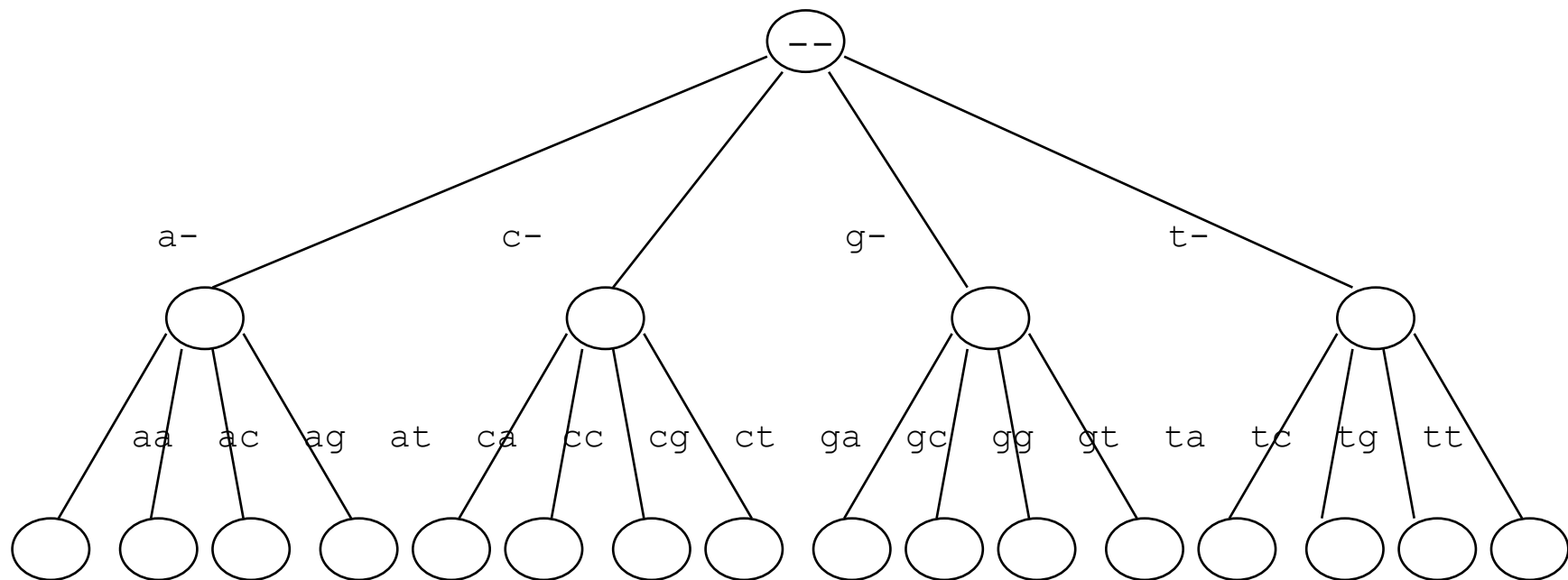
Search Trees

- Linked list is not the most efficient data structure for motif finding
- Let's try grouping the sequences by their prefixes



Search Trees (cont' d)

- Linked lists are a bad structuring of the sequences
- Let's try grouping the sequences by their prefixes



Analyzing Search Trees

- The paths from the initial node to any node is typically much shorter in a search tree than in a linked list
- Characteristics of the search trees:
 - The sequences are contained in its leaves
 - The parent of a node is the prefix of its children
- How can we move through the tree?

Moving through the Search Trees

- Once the tree is built, we need to design algorithms to move through the tree
- Four common moves in a search tree that we are about to explore:
 - Move to the next leaf
 - Visit all the leaves
 - Visit the next node
 - Bypass the children of a node

Traversing the Search Tree

- The data are contained in the leaves
- Furthermore, they are ordered
- We want to search leaves sequentially
- Given a current leaf, we need to compute the integer permutation of the “next” leaf

The Next Leaf

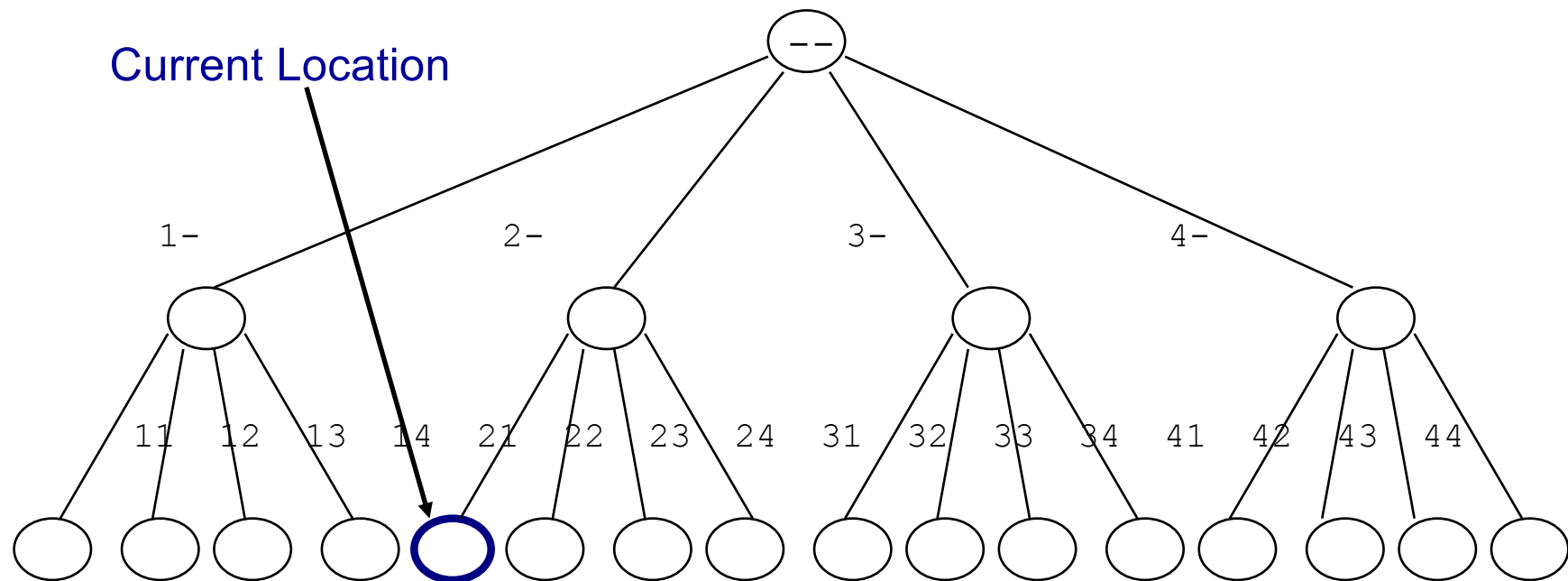
```
1. NextLeaf( a,  $\ell$ , k)           // a : the array of digits
2.   for  $i \leftarrow \ell$  to 1       //  $\ell$  : length of the array
3.     if  $a_i < k$                    // k : max digit value
4.        $a_i \leftarrow a_i + 1$ 
5.       return a
6.      $a_i \leftarrow 1$ 
7.   return a
```

The Next Leaf (cont' d)

- The algorithm is common addition in radix k
- Try to increment the least significant digit
- “Carry the one” to the next digit position when the digit is at maximal value

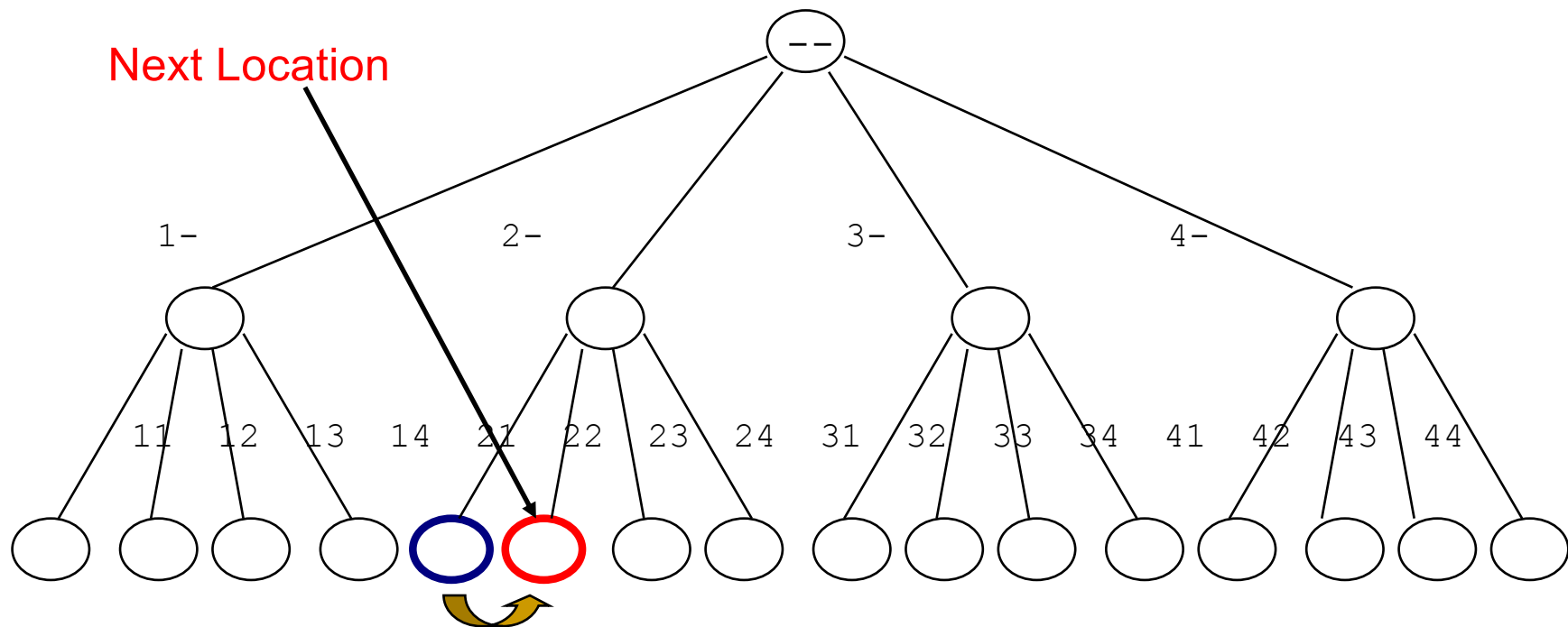
The Next Leaf: Example

- Moving to the next leaf:



The Next Leaf: Example (cont' d)

- Moving to the next leaf:



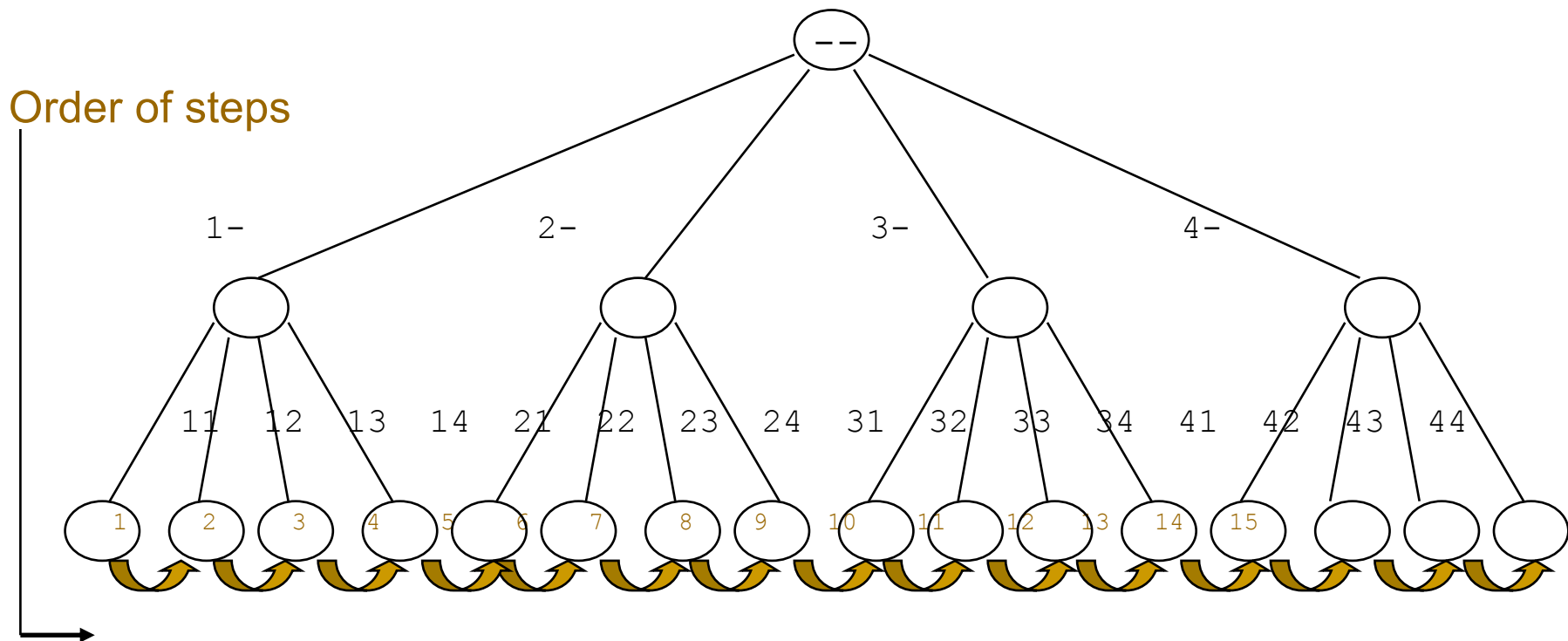
Visit All Leaves

- Printing all permutations in ascending order:

```
1.  AllLeaves( $\ell$   $k$ )  //  $\ell$ : length of the sequence
2.       $a \leftarrow (1, \dots, 1)$  //  $a$ : array of digits
3.      while true:    //  $k$ : max digit value
4.          output  $a$ 
5.           $a \leftarrow \text{NextLeaf}(a, \ell, k)$ 
6.          if  $a = (1, \dots, 1)$ 
7.              return
```

Visit All Leaves: Example

- Moving through all the leaves in order:



Depth First Search

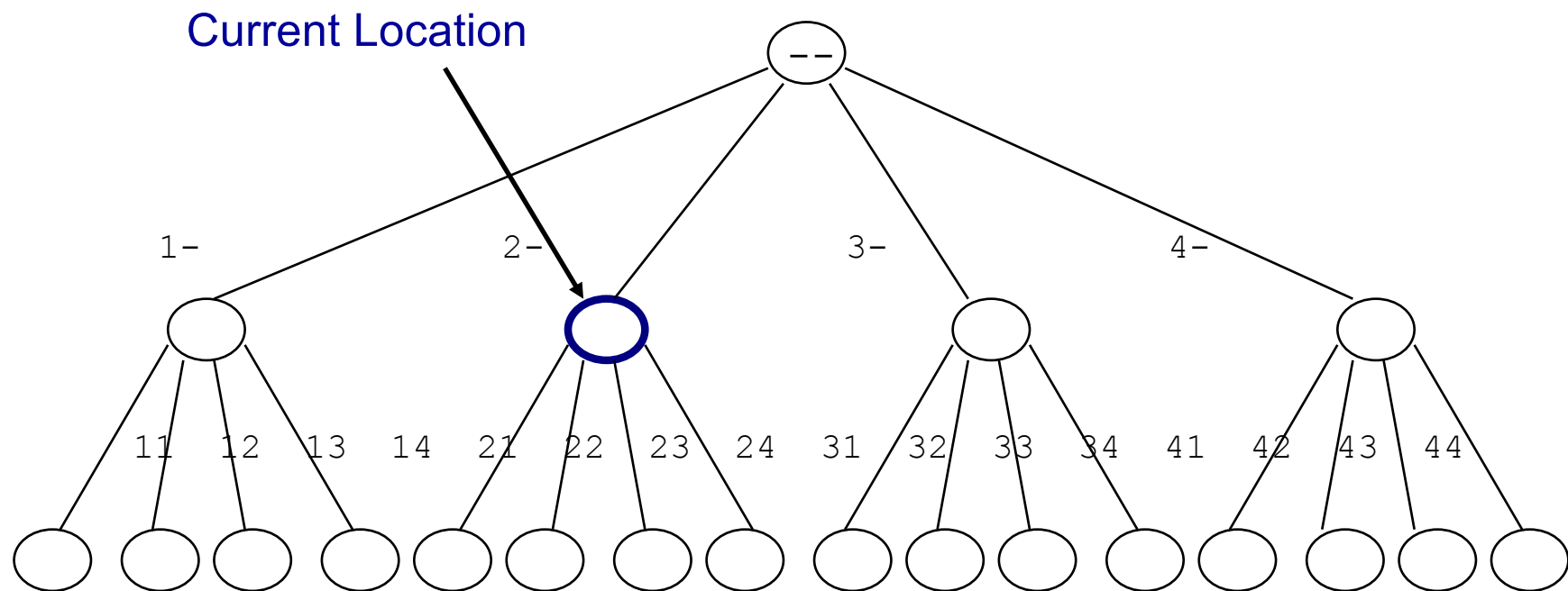
- So we can search leaves
- How about searching vertices sequentially?
- We can do this with a depth first search

Going to the Next Vertex

```
1. NextVertex(a, i, l, k) // a : the array of digits
2.   if  $i < l$  // i : length of sequence
3.      $a_{i+1} <---$  1 // l : max length
4.     return ( a, i+1 ) // k : max digit value
5.   else
6.     for  $j <---$  l to 1
7.       if  $a_j < k$ 
8.          $a_j <---$   $a_j + 1$ 
9.         return( a, j )
10.  return(a, 0)
```

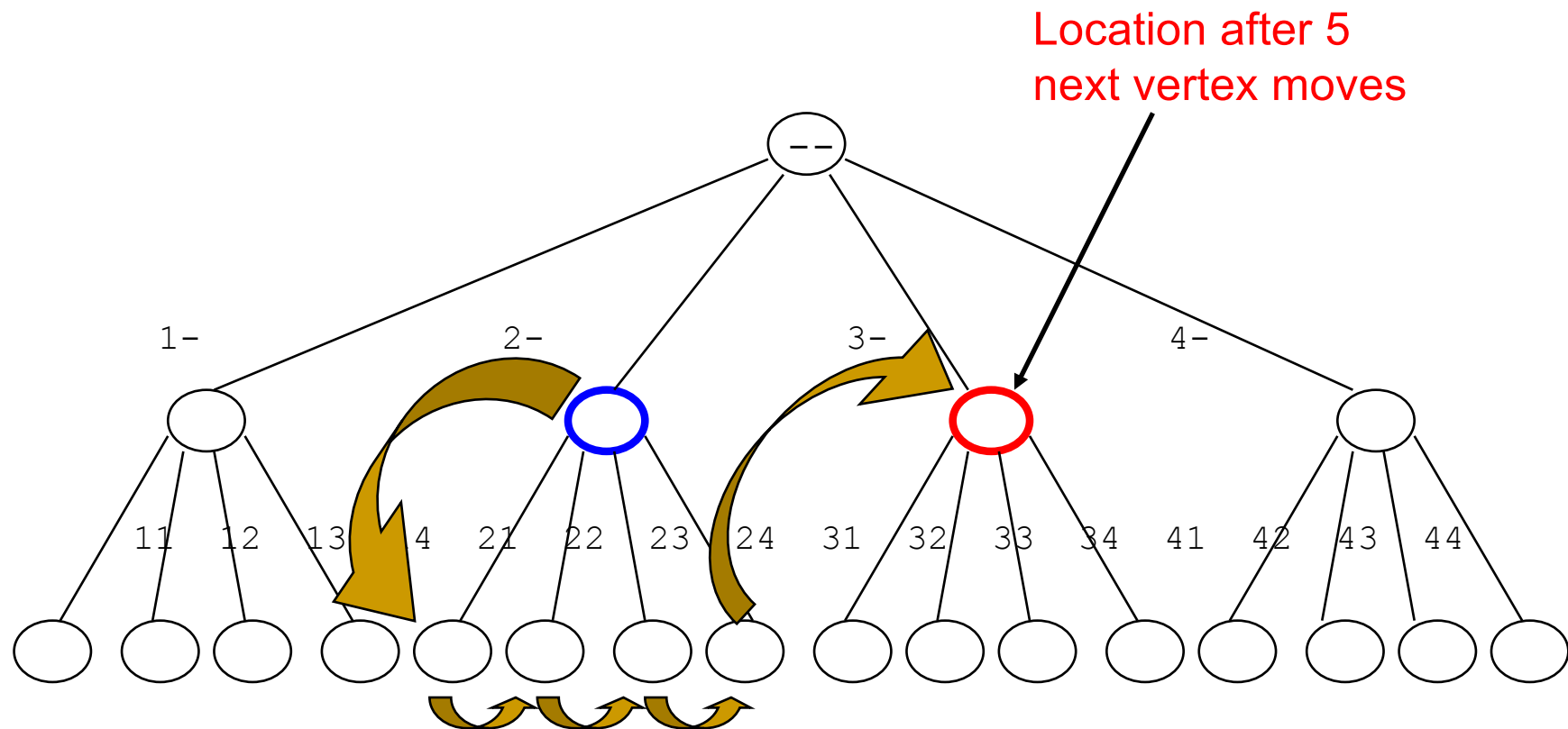
Example

- Moving to the next vertex:



Example

- Moving to the next vertices:



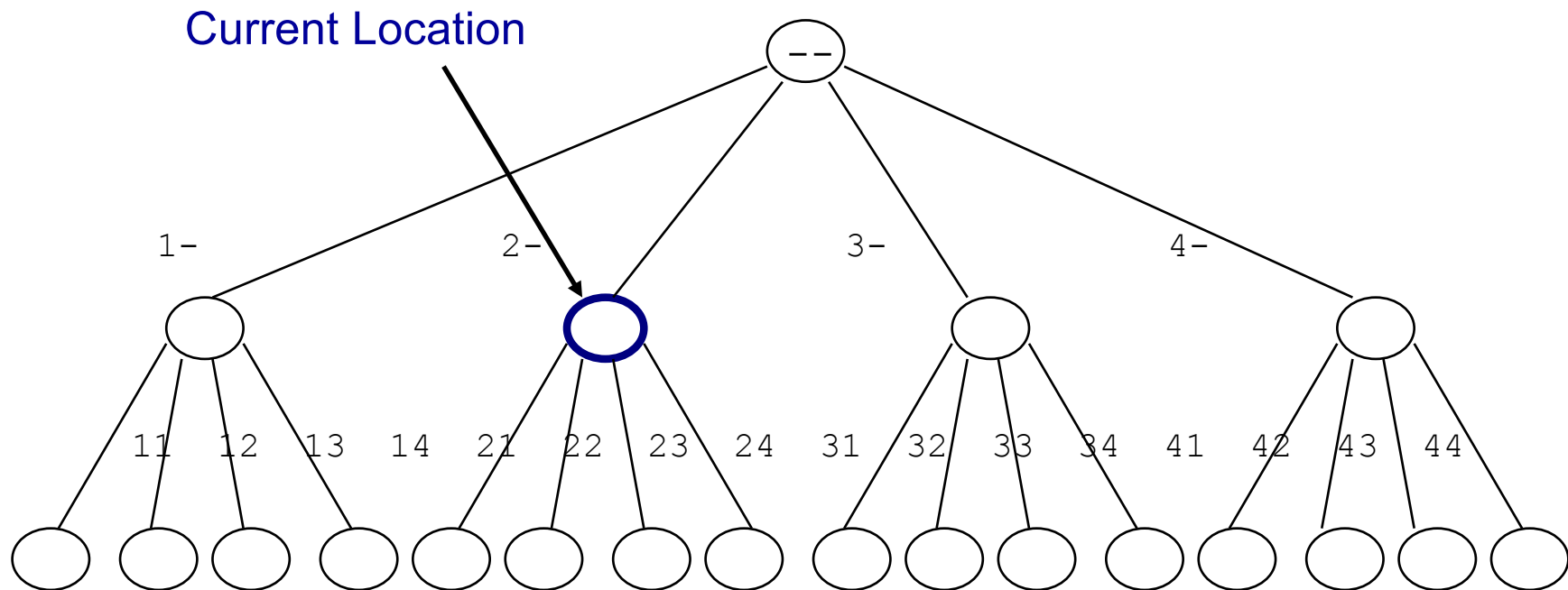
Bypass Move

- Given a prefix (internal vertex), find next vertex after skipping all its children

```
1.  ByPass(a, i, k)    // a: array of digits
2.    for  $j \leftarrow i$  to 1  // i: prefix length
3.      if  $a_j < k$         // k: maximum length
4.         $a_j \leftarrow a_j + 1$ 
5.      return(a, j)
6.    return(a, 0)
```

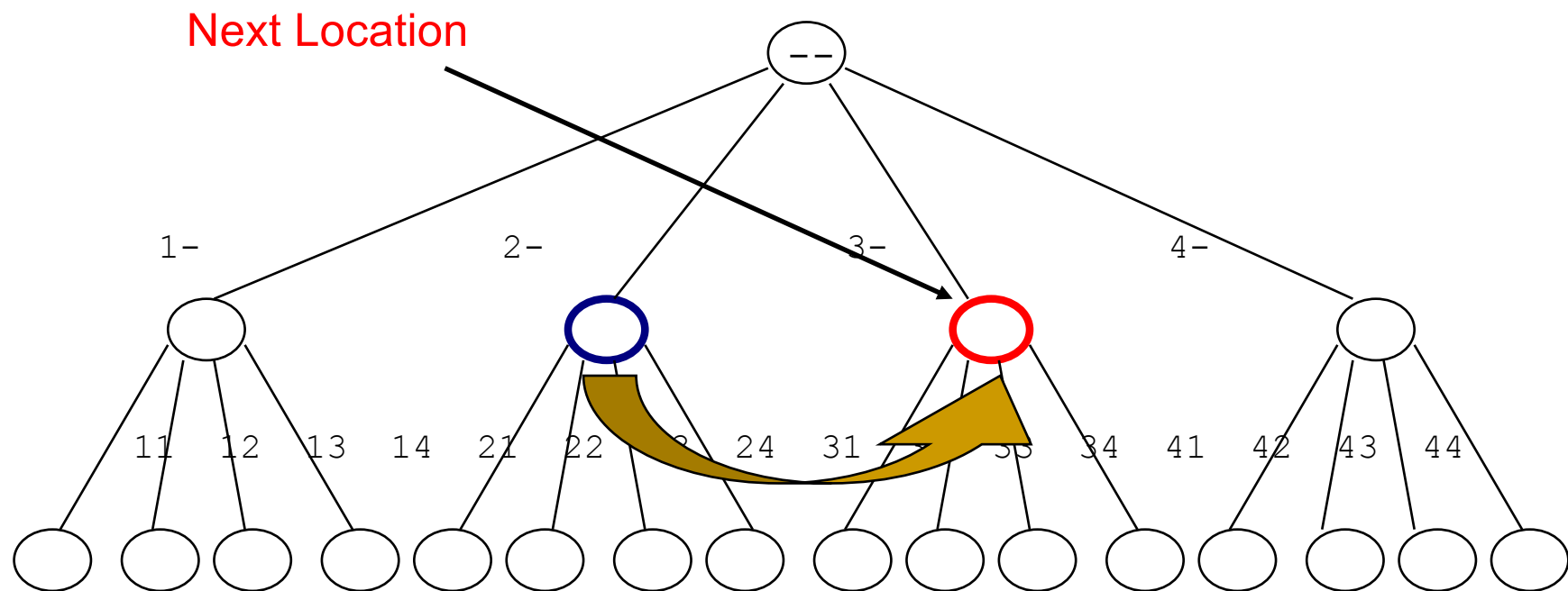
Bypass Move: Example

- Bypassing the descendants of “2-”:



Example

- Bypassing the descendants of “2-”:



Revisiting Brute Force Search

- Now that we have method for navigating the tree, lets look again at the Brute Force Motif Search

Brute Force Search Again

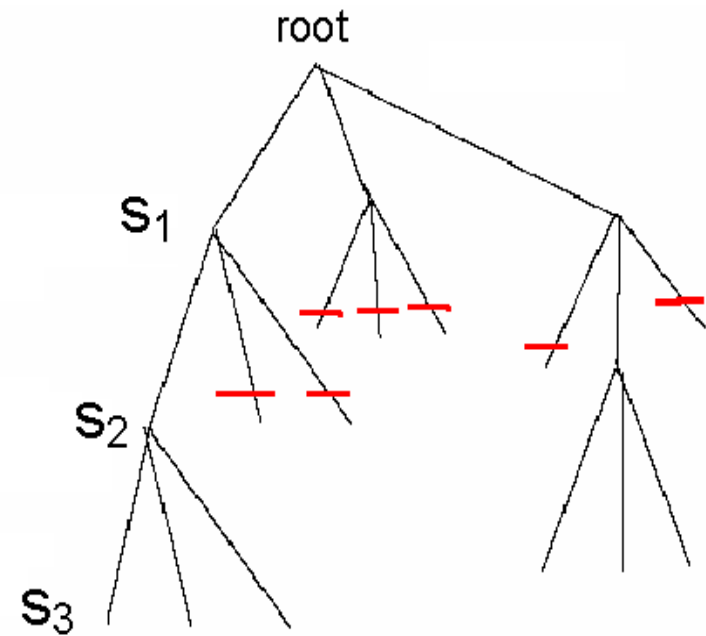
1. BruteForceMotifSearchAgain(*DNA*, *t*, *n*, ℓ)
2. $s \leftarrow (1, 1, \dots, 1)$
3. $bestScore \leftarrow Score(s, DNA)$
4. while forever
5. $s \leftarrow \text{NEXTLEAF}(s, t, n - \ell + 1)$
6. if ($Score(s, DNA) > bestScore$)
7. $bestScore \leftarrow score(s, DNA)$
8. $bestMotif \leftarrow (s_1, s_2, \dots, s_t)$
9. return $bestMotif$

Can We Do Better?

- Sets of $\mathbf{s}$ may have a weak profile for $(s_1, s_2, \dots, s_i)$
- Optimism: if all subsequent positions add to score
 - Contributes $(t - i) * \ell$ to $\text{Score}(\mathbf{s}, i, \mathbf{DNA})$
- Denotes $\text{Score}(\mathbf{s}, i, \mathbf{DNA})$ be the partial consensus score of $i \times l$ alignment matrix involving only the first i rows of the $\mathbf{DNA}$
- If $\text{Score}(\mathbf{s}, i, \mathbf{DNA}) + (t - i) * \ell < \text{BestScore}$, it makes no sense to search in vertices of the current subtree
 - Use **ByPass()**

Branch and Bound Algorithm for Motif Search

- Since each level of the tree goes deeper into search, discarding a prefix discards all following branches
- This saves us from looking at $(n - \ell + 1)^{t-i}$ leaves
 - Use **NextVertex()** and **ByPass()** methods described earlier to navigate the tree



Pseudocode for Branch and Bound Motif Search

```
1.  BranchAndBoundMotifSearch(DNA, t, n, l)
2.  S  $\leftarrow$  (1,...,1)
3.  bestScore  $\leftarrow$  0
4.  i  $\leftarrow$  1
5.  while i > 0
6.      if i < t
7.          optimisticScore  $\leftarrow$  Score(s, i, DNA) + (t - i) * l
8.          if optimisticScore < bestScore
9.              (s, i)  $\leftarrow$  Bypass(s, i, t, n - l + 1)
10.         else
11.             (s, i)  $\leftarrow$  NextVertex(s, i, t, n - l + 1)
12.         else
13.             if Score(s, DNA) > bestScore
14.                 bestScore  $\leftarrow$  Score(s)
15.                 bestMotif  $\leftarrow$  (s1, s2, s3, ..., st)
16.                 (s, i)  $\leftarrow$  NextVertex(s, i, t, n - l + 1)
17.  return bestMotif
```

Median String Search Improvements

- Recall the computational differences between motif search and median string search
 - The Motif Finding Problem needs to examine all the combinations for s . That is $(n-l+1)^l$ combinations
 - The Median String Problem needs to examine 4^l combinations of v . This number is relatively smaller
- We want to use median string algorithm with the Branch and Bound trick!

Branch and Bound Applied to Median String Search

- Note that if the total distance for a prefix is greater than that for the best word so far:

$$\text{TotalDistance}(\textit{prefix}, \textit{DNA}) + \text{ZERO} > \textit{BestDistance}$$

there is no use exploring the remaining part of the word

- We can eliminate that branch and **BYPASS** exploring that branch further

Bounded Median String Search

```
1. BranchAndBoundMedianStringSearch(DNA, t, n, l)
2.  $s \leftarrow (1, \dots, 1)$ 
3.  $bestDistance \leftarrow \infty$ 
4.  $i \leftarrow 1$ 
5. while  $i > 0$ 
6.   if  $i < l$ 
7.      $prefix \leftarrow$  nucleotide string of  $s$ 
8.      $optimisticDistance \leftarrow TotalDistance(prefix, DNA)$ 
9.     if  $optimisticDistance > bestDistance$ 
10.       $(s, i) \leftarrow Bypass(s, i, l, 4)$ 
11.   else
12.      $(s, i) \leftarrow NextVertex(s, i, l, 4)$ 
13.   else
14.      $word \leftarrow$  nucleotide string for  $s$ 
15.     if  $TotalDistance(word, DNA) < bestDistance$ 
16.        $bestDistance \leftarrow TotalDistance(word, DNA)$ 
17.        $bestWord \leftarrow word$ 
18.      $(s, i) \leftarrow NextVertex(s, i, l, 4)$ 
19. return  $bestWord$ 
```

More on the Motif Problem

- Exhaustive Search and Median String are both exact algorithms
- Meaning they always find the optimal solution, though they may be too slow to perform practical tasks
- Many algorithms today sacrifice optimal solution for speed

Seeds

- How often do we see the motif AAAAAAA in nature? Why start there?
- Often, initial k -mers are either randomly generated or taken from sequences. These are called **seeds**
- Sometimes seeds are generated in a greedy algorithm that compares a few of the sequences first before computing best distances

CONSENSUS: Greedy Motif Search

- Find two closest l -mers in sequences 1 and 2 and forms $2 \times l$ alignment matrix with $\text{Score}(\mathbf{s}, 2, \text{DNA})$
- At each of the following $t-2$ iterations CONSENSUS finds a “best” l -mer in sequence i from the perspective of the already constructed $(i-1) \times l$ alignment matrix for the first $(i-1)$ sequences
- In other words, it finds an l -mer in sequence i maximizing

$$\text{Score}(\mathbf{s}, i, \text{DNA})$$

under the assumption that the first $(i-1)$ l -mers have been already chosen

- CONSENSUS sacrifices optimal solution for speed: in fact the bulk of the time is actually spent locating the first 2 l -mers
-

Some Motif Finding Programs

- **Pattern and Profile Branching**
Price, Pevzner (2003)
- **MULTIPROFILER**
Keich, Pevzner (2002)
- **CONSENSUS**
Hertz, Stromo (1999)
- **MEME**
Bailey, Elkan (1995)
- **GibbsDNA**
Lawrence et al (1993)
- **RandomProjections**
Buhler, Tompa (2002)